

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Bionic Turtle. "FRM: Why We Use Log Returns in Finance." *YouTube*, 3 Feb. 2009, <https://www.youtube.com/watch?v=PtoUlt3V0CI>.

- **Required pages:** Whole video
- **Estimated reading time:** 6 minutes

2. Damodaran, Aswath. "Corporate Finance B40.2302 Lecture Notes: Packet 1." New York University, <https://people.stern.nyu.edu/adamodar/pdfiles/cfovhds/cfpacket1spr19.pdf>.

- **Required pages:** Slide numbers 74-76
- **Estimated reading time:** 10 minutes

3. "Measures of Spread: Range, Variance & Standard Deviation." *Khan Academy*, <https://www.khanacademy.org/math/statistics-probability/summarizing-quantitative-data/variance-standard-deviation-population/v/range-variance-and-standard-deviation-as-measures-of-dispersion>.

- **Required pages:** Whole video
- **Estimated reading time:** 12 minutes

Financial Asset Data

Reading Time 60 minutes

Prior Knowledge Basic Python, Algebra

Keywords Stock, Bitcoin, Bond Prices, ETFs, Simple Returns, Analyzing and Charting Data in Python

For the first lesson of this module, we will recap some of the concepts you learned during Financial Markets. We'll examine stocks, bonds, and cryptocurrency prices during this lesson. Finally, we will also demonstrate how to pull price data from Python and do some basic price analysis of equities, Bitcoin, and bond data. Then, we'll expand on what we learned with a few custom volatility metrics, along with how to program them in Python. We'll also focus on the risk, or volatility of returns, of financial assets.

In [1]: `!pip install yfinance --upgrade --no-cache-dir`

Requirement already satisfied: yfinance in /usr/local/lib/python3.11/site-packages (0.2.65)

Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.0.0)

Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.11/site-packages (from yfinance) (1.26.2)

Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.32.3)

Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.11/site-packages (from yfinance) (0.0.11)

Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (3.4.0)

Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.11/site-packages (from yfinance) (2023.3.post1)

Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.4.6)

Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.11/site-packages (from yfinance) (3.17.8)

Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.11/site-packages (from yfinance) (4.12.2)

Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.11/site-packages (from yfinance) (0.13.0)

Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (6.32.0)

Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (15.0.1)

Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.4.1)

Requirement already satisfied: cffi>=1.12.0 in /usr/local/lib/python3.11/site-packages (from curl_cffi>=0.7->yfinance) (1.15.1)

Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.11/site-packages (from curl_cffi>=0.7->yfinance) (2025.8.3)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2.8.2)

Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2023.3)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.1.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.4)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (1.26.15)

Requirement already satisfied: pycparser in /usr/local/lib/python3.11/site-packages (from cffi>=1.12.0->curl_cffi>=0.7->yfinance) (2.21)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.16.0)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.2

[notice] To update, run: `pip install --upgrade pip`

```
In [2]: import datetime
import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd
import seaborn as sns
import yfinance as yfin

pd.options.display.float_format = "{:,.6f}".format
```

1. Pull Equities Data for Amazon and Ford, along with Bitcoin Data

The following code pulls five years of equity price data for Ford and Amazon along with the daily Bitcoin prices over the same time period. The reason we chose Ford and Amazon here is to give examples from different sectors. While Amazon has been around for a while now, it still represents a newer, popular tech stock while Ford is very much part of the old guard, trading publicly since 1956.

This code snippet uses the `yfinance` library to download historical stock data for Amazon (AMZN), Ford (F), and Bitcoin (BTC-USD).

```
In [3]: # Starting and end dates
start = datetime.date(2019, 8, 1)
end = datetime.date(2024, 8, 1)

# Get Amazon, Ford and Bitcoin data
df = yfin.download(["AMZN", "F", "BTC-USD"], start, end, auto_adjust = False)["Adj"]

# Convert DataFrame index to timezone-aware (UTC)
df.index = df.index.tz_localize('UTC')

# Drop Nan
df.dropna(inplace=True)
```

```
[*****100%*****] 3 of 3 completed
```

1.1 Taking a Look at the Data

Unless you're extremely familiar with the dataset you're working with, it's almost always a good idea to look at the first few rows of data to get an idea of how it looks. We can accomplish this using the `head()` method. You can pass a numerical parameter in this (for example, 10), and it will return the top 10 rows of the data set. By default, it returns the first five.

```
In [4]: df.head(10)
```

Out[4]:

	Ticker	AMZN	BTC-USD	F
	Date			
	2019-08-01 00:00:00+00:00	92.765999	10,399.668945	6.912731
	2019-08-02 00:00:00+00:00	91.162003	10,518.174805	6.890456
	2019-08-05 00:00:00+00:00	88.256500	11,805.653320	6.853330
	2019-08-06 00:00:00+00:00	89.391502	11,478.168945	7.038958
	2019-08-07 00:00:00+00:00	89.669998	11,941.968750	7.076081
	2019-08-08 00:00:00+00:00	91.644501	11,966.407227	7.098357
	2019-08-09 00:00:00+00:00	90.378998	11,862.936523	7.016681
	2019-08-12 00:00:00+00:00	89.246002	11,382.616211	6.897880
	2019-08-13 00:00:00+00:00	91.217003	10,895.830078	6.875605
	2019-08-14 00:00:00+00:00	88.148003	10,051.704102	6.682554

Other methods are `df.tail(n)` with an argument indicating the last `n` rows of dataset or just running the `df` command to get the first 5 rows and last 5 rows of the dataset.

In our example above, right off the bat, we can see that it was a smart idea to take a look at this data first. Can you guess why we have null values here?

The answer is that simple equities only trade Monday through Friday, which means there's no price data available on the weekends. Bitcoin does not have this same characteristic as it trades seven days a week, which is why 2019-08-03 and 2019-08-04 are populated in this case.

1.2 Digging a Little Deeper into the Data

We can use the pandas `describe()` method to show summary stats of our data. We can see that Bitcoin has a much higher number of observations (count) than our two stocks, which, as just discussed, is because stocks are not traded on weekends. The other summary stats are relatively basic, like mean and standard deviation along with showing minimum, maximum, and a few quantiles. This might be something you're interested in, but it is hard to compare these investments with this data alone since each asset had a different starting price five years ago. For a better comparison, we will have to examine returns in more depth in an upcoming lesson.

In [5]: `df.describe()`

Out[5]:

Ticker	AMZN	BTC-USD	F
count	1,258.000000	1,258.000000	1,258.000000
mean	137.853120	30,864.423732	9.554362
std	31.730629	18,377.034296	2.907015
min	81.820000	4,970.788086	3.077781
25%	107.565500	13,465.042725	6.936322
50%	143.729996	28,067.854492	10.086829
75%	164.599251	43,722.896484	11.187366
max	200.000000	73,083.500000	19.431341

1.3 Charting Prices over 2023

If we ever want to get a quick plot of the DataFrame's data, we can use the aptly named `plot()` method. As shown below, to chart the 2023 returns of these three assets:

In [6]:

```

# Create the figure.
# We want a plot where the three assets have the same index (x-axis) but different
fig = plt.figure(figsize=(13,7))
ax1 = fig.add_subplot(111)
ax2 = ax1.twinx()
ax3 = ax1.twinx()

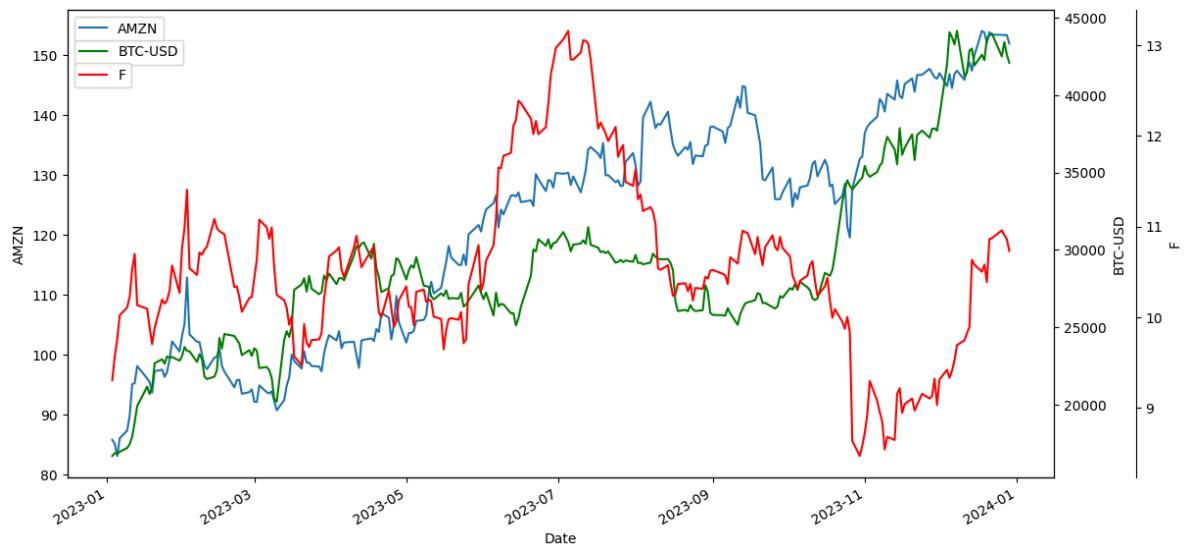
# Plot the data
df["2023-01-01":"2023-12-31"].plot(ax=ax1, y='AMZN', legend=True)
df["2023-01-01":"2023-12-31"].plot(ax=ax2, y='BTC-USD', legend=True, color='g')
df["2023-01-01":"2023-12-31"].plot(ax=ax3, y='F', legend=True, color='r')

# We set the labels to the axes
ax1.set_ylabel('AMZN')
ax2.set_ylabel('BTC-USD')
ax3.set_ylabel('F')
ax3.spines['right'].set_position(('outward', 60))

# Set position of legends
ax1.legend(['AMZN'], loc='upper left')
ax2.legend(['BTC-USD'], loc='upper left', bbox_to_anchor=(0, 0.95))
ax3.legend(['F'], loc='upper left', bbox_to_anchor=(0, 0.9))

plt.show()

```



The above code and chart demonstrate how we can easily pass in a date range to the plot method in order to chart a subset of the data. If no date range is supplied, by default, the plot will encapsulate the entire dataset. This code snippet creates a figure with three subplots that share the same x-axis (date) but have different y-axes to plot the prices of Amazon (AMZN), Bitcoin (BTC-USD), and Ford (F) for the year 2023. This approach allows us to visualize the price trends of three different assets with varying scales on the same plot. However, the chart also illustrates why using price alone is not ideal for comparing two assets. The scale of Bitcoin's price versus a low-price stock like Ford makes it very hard to compare the two when deciding which stock is better to invest in.

2. Calculating Return on Investment

If we invested **\$1,000** into each of these assets five years ago, how much money would we have today? To answer this question, we first need to determine how many shares of each stock we could buy with **\$1,000** at the start of our date range. For the purposes of this exercise, we will use **01/08/2019** as our starting point because i

From our data above, we can see the starting prices of AMZN, F, and Bitcoin. We can also have the values of the first row for each ticker stored in individual variables, which we can then use for further calculations or analysis. We will do this in the following code. We also need to divide **\$1,000** by each of these numbers to see how many shares we will have:

```
In [7]: # Access the first row of the DataFrame
first_row = df.iloc[0]

# Assign price values to variables
amzn_price = first_row["AMZN"]
f_price = first_row["F"]
btc_price = first_row["BTC-USD"]

# Print price values
```

```

print("Purchase price of AMZN:", np.round(amzn_price, 3))
print("Purchase price of F:", np.round(f_price, 3))
print("Purchase price of BTC-USD:", np.round(btc_price, 3))
print(" - - - - -")

# Divide $1,000 by each of price values to get number of shares
amzn_shares = 1000 / amzn_price
f_shares = 1000 / f_price
btc_shares = 1000 / btc_price

# Print number of shares for each ticker
print("Number of shares of AMZN:", np.round(amzn_shares, 3))
print("Number of shares of F:", np.round(f_shares, 3))
print("Number of shares of BTC-USD:", np.round(btc_shares, 3))

```

```

Purchase price of AMZN: 92.766
Purchase price of F: 6.913
Purchase price of BTC-USD: 10399.669
- - - - -
Number of shares of AMZN: 10.78
Number of shares of F: 144.661
Number of shares of BTC-USD: 0.096

```

It's a relatively recent phenomenon that retail brokers offer fractional shares, but this tends to be the case nowadays for most commonly used brokers. For this exercise, we will assume we can have fractional shares.

Now, to determine how much money we would have today, we look at the most recent date in our dataset (31/07/2024), get the prices of each asset, and multiply our number of shares by this number. We can find this by looking at the bottom of our dataset in using the *tail()* method. Alternatively, we can also program asset value computations:

```

In [8]: # Get last date values from df
last_row = df.iloc[-1]

# Assign end date price values to variables
amzn_price_end = last_row["AMZN"]
f_price_end = last_row["F"]
btc_price_end = last_row["BTC-USD"]

# Print end date prices
print("End price of AMZN:", np.round(amzn_price_end, 3))
print("End price of F:", np.round(f_price_end, 3))
print("End price of BTC-USD:", np.round(btc_price_end, 3))
print(" - - - - -")

# Compute end date values for each ticker
amzn_value = amzn_price_end * amzn_shares # need 20 times multiple due to stock sp
f_value = f_price_end * f_shares
btc_value = btc_price_end * btc_shares

# Print end date values
print("Holding value of AMZN:", np.round(amzn_value, 3))

```

```
print("Holding value of F:", np.round(f_value, 3))
print("Holding value of BTC-USD:", np.round(btc_value, 3))
```

```
End price of AMZN: 186.98
End price of F: 9.899
End price of BTC-USD: 64619.25
- - - - -
Holding value of AMZN: 2015.609
Holding value of F: 1432.036
Holding value of BTC-USD: 6213.587
```

The investment spend was \$1000 for each ticker. Now based on the computation results above, it seems that our investments have performed differently:

- AMZN: Our Amazon investment has more than doubled, indicating a significant gain.
- F: Our Ford investment has also grown, showing a decent return.
- BTC-USD: Our Bitcoin investment has increased by more than six times, suggesting a substantial profit.

However, it's important to remember that these are just the final values and don't tell the whole story. To get a clearer picture of investment performance, we need to consider:

- Timeframe: We need to know the timeframe of this investment to assess the returns accurately. Short-term fluctuations are normal, while long-term trends are more indicative of overall performance.
- External Factors: Market volatility, economic conditions, and company-specific news can significantly impact stock prices.
- Investment Strategy: Your individual investment goals and risk tolerance should guide your decisions.

These assets were relatively easy to compare here since we started with \$1,000 in each. If we had started with different values, we would need to calculate the returns for an apples-to-apples comparison. In a future lesson, we will go over simple and log returns, but for convenience, we will use simple returns:

Simple Returns Formula

$$R_{\text{simple}} = \frac{p_1 - p_0}{p_0}$$

where p_1 = final value, p_0 = initial value

This will be easy in our case since \$1,000 is the initial value in all three situations.

```
In [9]: # Compute simple returns
amzn_return = (amzn_value - 1000) / 1000
f_return = (f_value - 1000) / 1000
btc_return = (btc_value - 1000) / 1000

# Print the returns
print("Simple return of AMZN:", np.round(amzn_return * 100, 3), "%")
```

```
print("Simple return of F:", np.round(f_return * 100, 3), "%")
print("Simple return of BTC-USD:", np.round(btc_return * 100, 3), "%")
```

Simple return of AMZN: 101.561 %
 Simple return of F: 43.204 %
 Simple return of BTC-USD: 521.359 %

Bitcoin's return here is off the charts compared to the other two assets. Bitcoin is a relatively new — some would even say riskier — asset than the other two. Higher risk equals higher potential for rewards, and thus far, investing in Bitcoin has paid off handsomely.

3. Comparing Equities and Bitcoin to Bonds

Let's introduce one more asset class here: bonds. As you recall from the Financial Markets class, bonds not only have the return of principal, but they also return a coupon, usually annually or quarterly. To simplify things, we will use an exchange-traded fund (ETF), which tracks bonds. The ETF is the Vanguard Long-Term Bond Index Fund ETF or BLV for short. This fund intends to track the performance of the Bloomberg Barclays U.S. Long Government/Credit Float Adjusted Index. This index includes investment grade corporate, U.S. government, and international dollar-denominated bonds that have maturities greater than 10 years. At least 80% of the fund's total assets will be invested in bonds held in the index mentioned above (Vanguard).

The following code snippet will pull BLV data and join it with our current DataFrame, df.

```
In [10]: df = df.join(yfin.download(["BLV"], start, end, auto_adjust = False)["Adj Close"].t
df.rename(columns={"Adj Close": "BLV"}, inplace=True)

df
```

[*****100%*****] 1 of 1 completed

Out[10]:

	Ticker	AMZN	BTC-USD	F	BLV
	Date				
	2019-08-01 00:00:00+00:00	92.765999	10,399.668945	6.912731	75.507210
	2019-08-02 00:00:00+00:00	91.162003	10,518.174805	6.890456	76.035866
	2019-08-05 00:00:00+00:00	88.256500	11,805.653320	6.853330	76.687111
	2019-08-06 00:00:00+00:00	89.391502	11,478.168945	7.038958	77.200432
	2019-08-07 00:00:00+00:00	89.669998	11,941.968750	7.076081	77.330688
	...	...	...	...	...
	2024-07-25 00:00:00+00:00	179.850006	65,777.226562	10.210347	67.098373
	2024-07-26 00:00:00+00:00	182.500000	67,912.062500	10.237793	67.619766
	2024-07-29 00:00:00+00:00	183.199997	66,819.914062	10.073112	67.875725
	2024-07-30 00:00:00+00:00	181.710007	66,201.015625	9.917578	68.017929
	2024-07-31 00:00:00+00:00	186.979996	64,619.250000	9.899279	68.634132

1258 rows × 4 columns

3.1 Calculate Log Returns, Remove Unused Columns, and Drop Nulls

Before computing returns, we need to remove the nulls for the weekend dates.

```
In [11]: # In order to calculate the returns of the stocks, we need to drop the NA rows.
returns_stocks_BLV = df[['AMZN', 'F', 'BLV']].dropna().pct_change()

# Crypto is traded 24/7
returns_BTC = df[['BTC-USD']].pct_change()

# We broadcast the index of crypto on the stocks in order to join the datasets having the same index
returns_stocks = returns_stocks_BLV.reindex(returns_BTC.index)
returns = returns_BTC.join(returns_stocks_BLV, how = 'outer')[1:]
returns
```

Out[11]:

	Ticker	BTC-USD	AMZN	F	BLV
Date					
2019-08-02 00:00:00+00:00		0.011395	-0.017291	-0.003222	0.007001
2019-08-05 00:00:00+00:00		0.122405	-0.031872	-0.005388	0.008565
2019-08-06 00:00:00+00:00		-0.027740	0.012860	0.027086	0.006694
2019-08-07 00:00:00+00:00		0.040407	0.003115	0.005274	0.001687
2019-08-08 00:00:00+00:00		0.002046	0.022020	0.003148	0.003270
...		...	...	...	...
2024-07-25 00:00:00+00:00		0.006197	-0.005419	-0.183614	0.006685
2024-07-26 00:00:00+00:00		0.032456	0.014734	0.002688	0.007771
2024-07-29 00:00:00+00:00		-0.016082	0.003836	-0.016086	0.003785
2024-07-30 00:00:00+00:00		-0.009262	-0.008133	-0.015440	0.002095
2024-07-31 00:00:00+00:00		-0.023893	0.029002	-0.001845	0.009059

1257 rows × 4 columns

3.2 Show Summary Stats for the Returns

In [12]: `returns.describe()`

	Ticker	BTC-USD	AMZN	F	BLV
count	1,257.000000	1,257.000000	1,257.000000	1,257.000000	
mean	0.002305	0.000803	0.000671	-0.000034	
std	0.040880	0.022175	0.027760	0.009179	
min	-0.371695	-0.140494	-0.183614	-0.075169	
25%	-0.016073	-0.011068	-0.013605	-0.005253	
50%	0.000365	0.000643	0.000000	0.000572	
75%	0.020299	0.012222	0.014212	0.005206	
max	0.211097	0.135359	0.234414	0.049139	

3.3 Converting Daily Returns to Annual

In [13]: `(returns.describe()[["BTC-USD", "F"]])`

Out[13]:

	Ticker	BTC-USD	F
count	1,257.000000	1,257.000000	
mean	0.002305	0.000671	
std	0.040880	0.027760	
min	-0.371695	-0.183614	
25%	-0.016073	-0.013605	
50%	0.000365	0.000000	
75%	0.020299	0.014212	
max	0.211097	0.234414	

Since we are using simple returns, in order to properly annualize them, we will use the geometric mean:

$$R_{\text{Geom_Mean}} = \left(\prod_{i=1}^N \left(1 + r_i \right) \right)^{\frac{1}{N}} - 1$$

Then, the annualized returns will be given by:

$$R_{\text{Annualized_Returns}} = \left(R_{\text{Geom_Mean}} + 1 \right)^{252 \text{ or } 365} - 1$$

In [14]: *#Annualize BTC-USD with 365 trading days*

```
(np.prod(returns_BTC + 1, axis=0) ** (1/len(returns_BTC))) ** (365) - 1
```

Out[14]: Ticker
 BTC-USD 0.698959
 dtype: float64

In [15]: *#Annualize stocks with 252 trading days*

```
(np.prod(returns_stocks_BLV + 1, axis=0) ** (1/len(returns_stocks_BLV))) ** (252) -
```

Out[15]: Ticker
 AMZN 0.150742
 F 0.074584
 BLV -0.018936
 dtype: float64

We can run the same analysis on BLV as we did on the above stocks to get the total simple return over a five-year period using a slightly different method. This time, we'll calculate the simple return rate and then just multiply \$1,000 initial investment by `1 + returnRate`.

In [16]: *# Get initial and end prices from df*

```
blv_price = df.iloc[0]["BLV"]
blv_price_end = df.iloc[-1]["BLV"]

# Compute simple return rate
```

```

blv_return_rate = (blv_price_end - blv_price) / blv_price
print("Simple return rate of BLV:", np.round(blv_return_rate * 100, 3), "%")

# Compute total simple return
total_simple_return = 1000 * (1 + blv_return_rate)
print("Total simple return of BLV: $", np.round(total_simple_return, 3))

```

Simple return rate of BLV: -9.103 %
Total simple return of BLV: \$ 908.975

This output indicates that the investment in BLV has experienced a negative simple return of -9.103%. This means we've lost approximately 9.1% of our initial investment in BLV.

If we started with 1,000 USD, after five years, we would've ended up with $(1 - 9.103) * 1000 = 908.97$ USD.

This is a much lower return than what we would've gotten with Ford, Amazon, or Bitcoin. This is to be expected for the most part as bonds return a quarterly/annual coupon along with returning the principal. For this reason, bonds (especially government bonds) are considered safer assets than stocks or cryptocurrencies. One major thing to consider is if a company goes bankrupt, you may not get your principal back in a bond. That being said, the same goes for stocks. Bondholders also have less credit risk than stockholders: if you own a bond in Ford and your friend owns Ford stock, but Ford declares bankruptcy tomorrow, the bondholder will be paid off first in the bankruptcy proceedings and the equity owner will only get what's left over, making bonds a safer investment.

The downside to this is that it's hard to achieve the level of total returns that a stock or cryptocurrency can achieve with a bond. There is a risk-reward tradeoff. Think back to the reading in this lesson for a moment. What we just discussed here is mentioned in the reading comparing a high-variance investment to a low-variance investment. What you choose is entirely up to your risk preferences. Always take into consideration the risk-reward tradeoff.

Here are some points to consider:

- **Time Horizon:** The simple return doesn't account for the investment timeframe. Short-term losses are not unusual, and it's important to evaluate investments over a longer period to get a more accurate picture of their performance.
- **Benchmarking:** Compare the return of BLV to a relevant benchmark, such as a broad market index like the S&P 500, to assess its relative performance.
- **Investment Strategy:** Consider your overall investment goals and risk tolerance when evaluating the performance of BLV.

Remember that simple returns are just one way to measure investment performance. For a more comprehensive analysis, consider factors like dividends, taxes, liquidity, regulation, credit risk, volatility, and the time value of money.

4. Future Value of an Investment

The S&P 500 consists of 500 large-cap companies. The index calculates weights according to market capitalization. This weighting is different from that in the Dow Jones Industrial Average, which is price weighted. The companies that make up this index are chosen by a committee, but companies must fit specific criteria before being included (Standard & Poor's):

- Market capitalization must be greater than or equal to \$18 billion.
- Annual dollar value traded to float-adjusted market capitalization is greater than 1.0.
- The minimum monthly trading volume is 250,000 shares in each of the six months leading up to the evaluation date.
- The company must be publicly listed on either the New York Stock Exchange (NYSE), including NYSE Arca or NYSE American, or NASDAQ (NASDAQ Global Select Market, NASDAQ Select Market or the NASDAQ Capital Market).
- The company should be from the U.S.
- Securities that are ineligible for inclusion in the index are limited partnerships, master limited partnerships and their investment trust units, OTC Bulletin Board issues, closed-end funds, exchange-traded funds, exchange-traded notes, royalty trusts, tracking stocks, preferred stock, unit trusts, equity warrants, convertible bonds, investment trusts, American depositary receipts, and American depositary shares.
- Since 2017, companies with dual share classes, like Standard & Poor's, are not added to the index.

This will be used as our proxy for large-cap stocks.

The Russell 2000 is an index that contains 2,000 small-cap companies and is frequently used as a benchmark by small-cap mutual funds for comparison purposes. The Russell 2000 is constructed to provide a barometer for small-cap stocks. It's reconstituted annually to ensure companies from the previous year don't get too large and distort the characteristics of small-caps (Maginn 118).

The Russell 2000 Index will be used as our proxy for small-cap companies.

One way to compare two investment opportunities is to determine the future value of an investment. There are always risks and many assumptions that need to be made when determining future value, especially for a risky investment like a stock. We will begin by using the average daily rate of return of each index to determine expected return. This is a simple way of thinking because it's impossible to say the next 10 years will be the same as the past 10 years, but it is a starting point when comparing investments. We will delve further into more advanced comparison metrics as the course goes on.

The formula for compound interest is:

$$FV = PV (1+i)^n$$

where

FV = Future value PV = Present value i = Periodic interest rate n = Number of periods

4.1 Pull 5 years daily price data for S&P 500 and Russell 2000

We're going to dig deeper into specific equities asset classes and compare them as potential investments. We will start by comparing large cap to small cap stocks using S&P 500 and the Russell 2000 Index.

Throughout this lesson we will build towards a function here which takes a date range and compares two series of returns.

```
In [17]: # Uncomment the following lines to pull most recent data (optional)
#start = datetime.date.today()-datetime.timedelta(365*10)
#end = datetime.date.today()
start = datetime.date(2019, 8, 1)
end = datetime.date(2024, 8, 1)

# Get data
prices = yfin.download(["^GSPC", "^RUT"], start, end, auto_adjust = False)["Adj Clo

# Rename column to make names more intuitive
prices = prices.rename(columns={"^GSPC": "SP500", "^RUT": "Russell2000"})

# Get dataset stats
prices.describe()
```

[*****100%*****] 2 of 2 completed

```
Out[17]:
```

	Ticker	SP500	Russell2000
count		1,258.000000	1,258.000000
mean		4,028.833253	1,865.957234
std		685.682781	284.232353
min		2,237.399902	991.159973
25%		3,502.235046	1,674.337524
50%		4,110.745117	1,873.654968
75%		4,468.547485	2,067.780090
max		5,667.200195	2,442.739990

```
In [18]: # Display first 5 rows
```

```
prices.head()
```

```
Out[18]:
```

	Ticker	SP500	Russell2000
	Date		
	2019-08-01	2,953.560059	1,550.760010
	2019-08-02	2,932.050049	1,533.660034
	2019-08-05	2,844.739990	1,487.410034
	2019-08-06	2,881.770020	1,502.089966
	2019-08-07	2,883.979980	1,500.689941

Once again, we can see these datasets have different starting points, so the only way we can do a fair comparison is by comparing returns.

4.2 Calculate Log Returns, Remove Unused Columns, and Drop Nulls

No nulls here for the weekend dates. Since the S&P 500 and the Russell 2000 Index are both stock indices, the weekends are automatically removed. The following code calculates the log returns of these financial assets:

```
In [19]: # Calculate log returns
df = np.log(prices) - np.log(prices.shift(1))
df = df.iloc[1:, 0:]
df.head()
```

```
Out[19]:
```

	Ticker	SP500	Russell2000
	Date		
	2019-08-02	-0.007309	-0.011088
	2019-08-05	-0.030230	-0.030621
	2019-08-06	0.012933	0.009821
	2019-08-07	0.000767	-0.000932
	2019-08-08	0.018588	0.020734

Log returns are often preferred over simple returns in financial analysis for several reasons:

- Time additivity: Log returns can be easily added across time to calculate cumulative returns over longer periods.
- Normality: Log returns tend to be more normally distributed than simple returns, which is a desirable property for statistical modeling.

- Symmetry: Log returns offer better symmetry than simple returns, especially for smaller changes, which makes them more suitable for certain types of financial analysis.

By calculating log returns, you can perform more sophisticated financial analysis and modeling.

4.3 Calculating Future Value of Each Index

In the following code snippet, we calculate the average annual return for each index using the mean of the daily returns multiplied by 252 (trading days per year). Then, we use these average returns to calculate future values.

```
In [20]: # Calculate average annual return over the past 5 years
avg_returns = df.mean() * 252

# Calculate future value for each index
future_value_sp500 = 1000 * np.exp(avg_returns["SP500"] * 5)
future_value_russell2000 = 1000 * np.exp(avg_returns["Russell2000"] * 5)

print("Future Value of SP500:", future_value_sp500)
print("Future Value of Russell2000:", future_value_russell2000)
```

Future Value of SP500: 1872.5041981875988

Future Value of Russell2000: 1455.0892174400183

Remember that past performance is not indicative of future results. Using the average return of the past 5 years is just one approach, and you should consider other factors and potential risks when making investment decisions.

In our specific case above, since we are calculating the future value over a 5-year period, using the average annual return is likely more suitable and efficient. However, if we were analyzing the short-term fluctuations or risk of the indices, daily returns would be more informative.

Whether it's better to calculate the average daily return or the average annual return depends on the context and how we plan to use the results. Here's a breakdown of the considerations:

- Comparability: If we want to compare the returns of different assets or investments with varying timeframes, annualized returns are generally more useful. Daily returns can be misleading if the investment periods are not the same.
- Future Value Calculation: If we're specifically calculating future values, using the average annual return directly in the formula is more straightforward and avoids extra steps.
- Volatility Analysis: If we're interested in analyzing the volatility or risk of an investment, daily returns might be more relevant as they provide more granular data.

Rewriting the previous code using daily returns is as follows:

```
In [21]: # Calculate average daily return over the past 5 years
avg_daily_returns = df.mean()

# Calculate future value for each index (using daily returns and 5 years)
num_days = 252 * 5 # Number of trading days in 5 years
future_value_sp500 = 1000 * np.exp(avg_daily_returns["SP500"] * num_days)
future_value_russell2000 = 1000 * np.exp(avg_daily_returns["Russell2000"] * num_days)

print("Future Value of SP500:", future_value_sp500)
print("Future Value of Russell2000:", future_value_russell2000)
```

```
Future Value of SP500: 1872.5041981875988
Future Value of Russell2000: 1455.0892174400183
```

This approach uses daily returns instead of annualized returns for the future value calculation. Remember that this assumes continuous compounding, which might not be entirely accurate for real-world investments.

The output shows the future value of an initial investment of \$1,000 in both the S&P 500 and Russell 2000 indices after 5 years, based on their average daily returns.

- S&P500: The future value is approximately \$1,872.50.
- Russell2000: The future value is approximately \$1,455.09.

While future value is important when comparing investments, we also need to be concerned about how volatile these investment opportunities are. In the next section, we will discuss three different volatility metrics along with how to program them in Python.

5. Investment Opportunities: Volatility

5.1 Price Volatility: High-Low

We will be using the prices DataFrame again to show some simple ways to compare the volatility of certain investments. The first method is comparing the high and low of the index. For this, we use the `max()` and `min()` methods to get the high and low over the duration of the DataFrame by column. From here, we can subtract these from each other to get an idea of the potential volatility of each investment.

```
In [22]: prices.max()
```

```
Out[22]: Ticker
SP500      5,667.200195
Russell2000 2,442.739990
dtype: float64
```

```
In [23]: prices.min()
```

```
Out[23]: Ticker
          SP500      2,237.399902
          Russell2000  991.159973
          dtype: float64
```

```
In [24]: prices.max() - prices.min()
```

```
Out[24]: Ticker
          SP500      3,429.800293
          Russell2000  1,451.580017
          dtype: float64
```

While some may find this useful, we definitely need to improve upon this metric to make it more useful when comparing different investments. How do you think we can do this? Let's try out two ideas.

The first is changing the timeframe slightly. Above, we're looking at the last 5 years of data. Perhaps it would be more useful to analyze more recent data. For this, we will look at the same metric over the last year:

```
In [25]: # Slice last year from prices data
end = pd.Timestamp(end).tz_localize(None) # Remove timezone info

currYear = prices.loc[
    pd.Timestamp(end - datetime.timedelta(365)) : end
]

# Calculate the difference between the maximum and minimum values in the currYear
currYear.max() - currYear.min()
```

```
Out[25]: Ticker
          SP500      1,549.830078
          Russell2000  626.729980
          dtype: float64
```

Even when comparing data over the last 365 days, it seems hard to do a comparison of these two datasets since they have different starting points. One final adjustment to this volatility metric could be to standardize it by dividing by the current price of the index:

```
In [26]: (currYear.max() - currYear.min()) / prices.iloc[-1]
```

```
Out[26]: Ticker
          SP500      0.280649
          Russell2000  0.277993
          dtype: float64
```

Volatility is a measure of the dispersion or fluctuation of returns. Higher volatility indicates greater price swings and potentially higher risk. Now that we have the data standardized by the current price, this high/low metric actually shows that we have similar daily volatility levels in both indices, with the S&P 500 being slightly more volatile than the Russell 2000.

Important Note:

- These values are based on historical data and may not reflect future volatility.
- Volatility can be calculated using different methods and timeframes. Make sure you understand the specific calculation used to interpret the results accurately.
- Understanding volatility is crucial for assessing investment risk and making informed decisions.

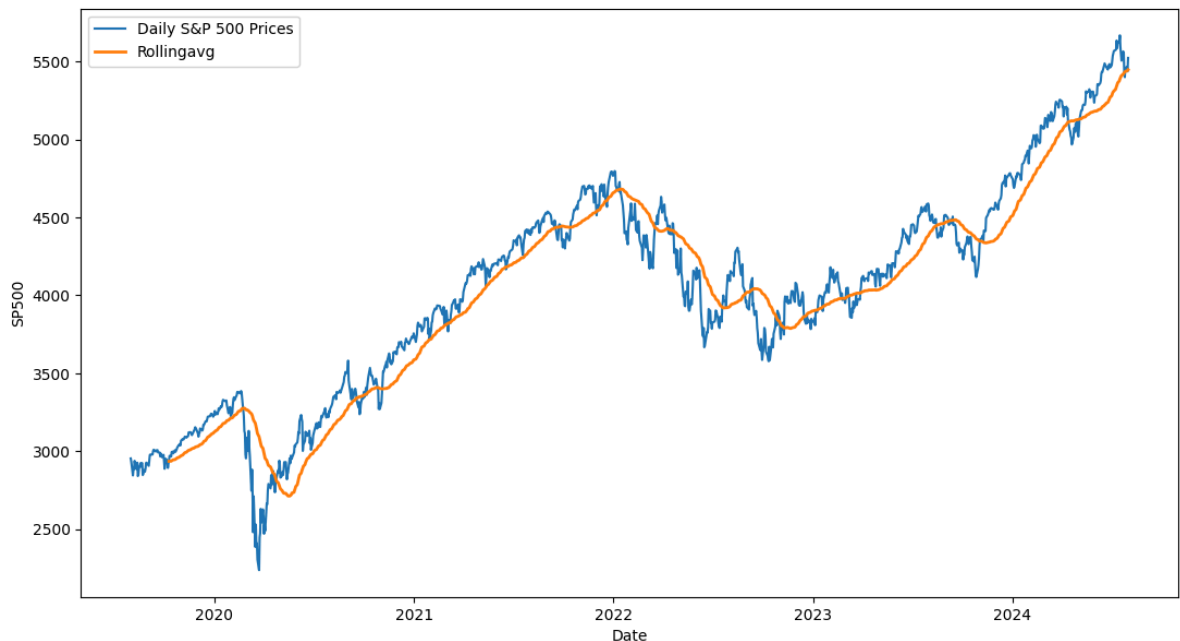
5.2 Moving Averages

We are going to create a volatility metric that compares each day's price to the moving average. The 50- and 200-day averages are commonly used when comparing investments. For our metric, we will use the 50-day average. Here's an example of how we can chart the S&P 500 along with its 50-day moving average:

```
In [27]: # Calculate the 50-day rolling average of the "SP500" column
prices["SP500 50 day_rolling_avg"] = prices.SP500.rolling(50).mean()

# set figure size and plot a simple time series plot using seaborn.lineplot()
plt.figure(figsize=(13, 7))
sns.lineplot(x="Date", y="SP500", data=prices, label="Daily S&P 500 Prices")

# plot rolling average
sns.lineplot(x="Date", y="SP500 50 day_rolling_avg", data=prices, label="Rollingavg")
plt.show()
```



This visualization helps to smooth out short-term fluctuations in daily prices and highlight the longer-term trend of the S&P 500. The rolling average provides a clearer picture of the overall direction of the market.

5.3 Rolling Distances

We can create a volatility metric here using this moving average. Let's imagine we're taking the distance between the moving average line and each data point from the S&P 500 chart. We can use this as a proxy for volatility. We also would want to treat the negative differences the same as the positive differences; otherwise, they may cancel each other out. As such, we will use the absolute value of each point. We will divide these differences by the prices in order to standardize these values, and finally, we'll take the average of those values. This can be programmed in Python with this simple line:

```
In [28]: ((abs(prices - prices.rolling(50).mean())) / prices).mean()
```

```
Out[28]: Ticker
SP500          0.037068
Russell12000    0.048725
SP500 50 day_rolling_avg  0.028209
dtype: float64
```

The result of this calculation provides a measure of how much, on average, the daily prices deviate from their 50-day rolling average, expressed as a percentage. This can be a useful indicator of the volatility or stability of the asset. Higher values suggest greater fluctuations around the rolling average, while lower values indicate more stable prices.

This is a different metric than the high/low metric we previously calculated, and it shows that the average deviation for the Russell 2000 is more volatile compared to the S&P 500. And the rolling average itself (SP500 50 day_rolling_avg) has an average deviation of 2.82%. This is expected as the rolling average is smoother than the daily prices.

These values can be useful for understanding the relative volatility of different assets and how much daily prices tend to deviate from their longer-term trends.

5.4 Price Volatility: Standard Deviations

Standard deviation is potentially the most popular volatility metric used when looking at an investment opportunity. It's as simple as calling the `std()` method on a DataFrame.

```
In [29]: prices.std()
```

```
Out[29]: Ticker
SP500          685.682781
Russell12000    284.232353
SP500 50 day_rolling_avg  635.644388
dtype: float64
```

As mentioned before, calling this on the prices is not as intuitive as calling this on the returns since prices start at different points. For a better comparison, we will call this `std()` method on the daily returns of the last 5 years:

```
In [30]: df.std()
```

```
Out[30]: Ticker
SP500      0.013458
Russell2000 0.017503
dtype: float64
```

Here again, the Russell 2000 has a slightly higher standard deviation than the S&P 500, suggesting that it has been more volatile over the last 5 years.

5.5 Writing a Volatility Comparison Function in Python

To tie it all together, we're going to take the three volatility metrics we've used up to this point, along with average daily return, and write a function, which takes:

- `startTime` - `dateTime` format
- `endTime` - `dateTime` format
- `tickers` - a dictionary of values where the key is the yahoo ticker and the value is the display name i.e. {"^GSPC": "SP500", "^RUT": "Russell2000"}

By writing a function like this, it allows our research to be reproducible and applicable to many different parameters. In our case, it will be a date range and dictionary of tickers. If you were writing a serious application, you would need to have lots of error handling in this function, such as guaranteeing that parameters passed in are of the correct type. In our case, we will assume the data types of parameters are correct and that the date range is longer than 365 days.

```
In [31]: def investCompare(startTime, endTime, tickers):
# pull price data from yahoo -- (list(tickers.keys())) = ['^GSPC', '^RUT']
startTime = pd.Timestamp(startTime).tz_localize('UTC')
endTime = pd.Timestamp(endTime).tz_localize('UTC')
prices = yfin.download(list(tickers.keys()), startTime, endTime, auto_adjust =
prices = prices.rename(columns=tickers)
returns = np.log(prices) - np.log(prices.shift(1))
returns = returns.iloc[1:, 0:]

# pull data into separate DataFrame to just look at the last 365 days of
# data for calculating our high/low metric
currYear = prices.loc[(endTime - datetime.timedelta(365)).tz_convert(prices.ind

# High-Low
highLow = (currYear.max() - currYear.min()) / prices.iloc[-1]
highLow = pd.DataFrame(highLow, columns=["HighMinusLow"])

# Moving average volatility
MA = pd.DataFrame(
    ((abs(prices - prices.rolling(50).mean())) / prices).mean(),
    columns=["MovingAverageVolatility"],
)
investments = pd.merge(highLow, MA, left_index=True, right_index=True)

# Standard deviation
investments = pd.merge(
```

```

        investments,
        pd.DataFrame(returns.std(), columns=["StandardDeviation"]),
        left_index=True,
        right_index=True,
    )

    # Daily return
    investments = pd.merge(
        investments,
        pd.DataFrame(returns.mean(), columns=["Daily Return Percentage"]),
        left_index=True,
        right_index=True,
    )

    # Format columns:
    # High-Low column rounded to 5 decimal
    investments["HighMinusLow"] = investments["HighMinusLow"].round(4).astype(str)

    # MA, std and returns columns as percentage
    columns = ["MovingAverageVolatility", "StandardDeviation", "Daily Return Percentage"]
    for col in columns:
        investments[col] = (investments[col] * 100).round(3).astype(str) + '%'

    return investments

```

Here we also format values: High-Low values are rounded to 4 decimal places and the values in the 'MovingAverageVolatility,' 'StandardDeviation,' and 'Daily Return Percentage' columns are formatted as percentages with 3 decimal rounding, which is good practice to facilitate comparison.

5.6 Running Our Volatility Function

We will start by calling our function with the two indices we've been examining thus far: the S&P 500 alongside the Russell 2000 Index. As we've seen, the three volatility metrics showed the Russell 2000 to be the more volatile investment. The average daily return is also higher for the S&P 500. Therefore, over the last 5 years, not only has the S&P 500 returned better than the Russell 2000, but it was also less volatile. Keep in mind that these different volatility metrics all calculate in some way how turbulently the stock has behaved in the past. This exercise was intended to demonstrate different ways of calculating volatility and how you can be a bit creative with it. This doesn't mean that volatility is constant or easily predictable, behaving the same way going forward; this exercise only reflects the past.

```

In [32]: investCompare(
        datetime.date(2019, 8, 1),
        datetime.date(2024, 8, 1),
        {"^GSPC": "SP500", "^RUT": "Russell2000"},
    )

```

[*****100%*****] 2 of 2 completed

Out[32]:

	HighMinusLow	MovingAverageVolatility	StandardDeviation	Daily Return Percentage
Ticker				
SP500	0.2806	3.707%	1.346%	0.05%
Russell2000	0.278	4.873%	1.75%	0.03%

This function gives us one clean space to compare investments in one line.

We also want to show that we made the function flexible in order to add symbols in the future. In the example below, we added Apple to the DataFrame in order to compare it to the other two indices.

In [33]:

```
investCompare(  
    datetime.date(2019, 8, 1),  
    datetime.date(2024, 8, 1),  
    {"^GSPC": "SP500", "^RUT": "Russell2000", "AAPL": "Apple"},  
)
```

[*****100%*****] 3 of 3 completed

Out[33]:

	HighMinusLow	MovingAverageVolatility	StandardDeviation	Daily Return Percentage
Ticker				
Apple	0.3154	6.462%	2.008%	0.118%
SP500	0.2806	3.707%	1.346%	0.05%
Russell2000	0.278	4.873%	1.75%	0.03%

These metrics show Apple to be a better investment in terms of daily return, but they also show that Apple is significantly more volatile than the two indices. This is expected, since the indices represent vast baskets of stocks. With many stocks moving in different directions, you'd expect the indices to be less volatile than an individual stock, even a blue-chip stock like Apple.

6. Comparing Growth vs. Value Stocks

We can continue using the function we just wrote to examine and compare different categories of investments, starting here with growth vs. value stocks. As a proxy for growth stocks, we will use the Vanguard Growth ETF (VUG). VUG is one of the biggest growth ETFs with around \$87 billion in assets under management.

For value stocks, we will use the Vanguard Value ETF (VTV). VTV is the most prominent value ETF with \$88 billion in assets under management. As you may imagine by now, Vanguard is a

popular purveyor of ETFs and has some of the lowest expense ratios in the industry.

```
In [34]: # Set start and end dates
start = datetime.date(2022, 8, 1)
end = datetime.date(2024, 8, 1)

# Compute investments stats using investCompare() function
investCompare(start, end, {"VUG": "Growth", "VTV": "Value"})
```

```
[*****100%*****] 2 of 2 completed
```

```
Out[34]:
```

	HighMinusLow	MovingAverageVolatility	StandardDeviation	Daily Return Percentage
Ticker				
Value	0.2306	2.546%	0.825%	0.05%
Growth	0.3582	4.083%	1.284%	0.078%

Running `investCompare` over the last two years for these ETFs shows that growth stocks have clearly outperformed value stocks in terms of returns — but not without increased volatility.

We need to keep in mind that timeframes have a large influence on our results here. Let's take a look at the data since 2010:

```
In [35]: # Set start and end dates
start = datetime.date(2010, 8, 1)
end = datetime.date(2024, 8, 1)

# Compute investments stats using investCompare() function
investCompare(start, end, {"VUG": "Growth", "VTV": "Value"})
```

```
[*****100%*****] 2 of 2 completed
```

```
Out[35]:
```

	HighMinusLow	MovingAverageVolatility	StandardDeviation	Daily Return Percentage
Ticker				
Value	0.2306	2.753%	1.043%	0.045%
Growth	0.3582	3.435%	1.224%	0.059%

Interestingly, compared to the previous analysis over a shorter timeframe, the differences in volatility and returns between Growth and Value seem to have narrowed slightly over this longer period. This could be due to the fact that, over the long term, the risk-return profiles of these investment styles are more balanced.

7. Comparing Domestic vs. Foreign Stocks

We will use the S&P 500 once again for U.S. stocks while finding an ETF for the foreign stocks. There are tons of options to choose from in this space.

The first question you need to ask yourself is where do you want to invest? It could be emerging markets, Europe, China, etc. To make things simple, we will use one example that is specific to Europe, ETF: SPDR Portfolio Europe ETF (SPEU), and one example that is specific to China, ETF: SPDR S&P China ETF (GXC).

```
In [36]: # Comparing Domestic vs. Foreign Stocks
start = datetime.date(2022, 8, 1)
end = datetime.date(2024, 8, 1)
investCompare(start, end, {"^GSPC": "SP500", "SPEU": "Europe ETF", "GXC": "China ETF"}
```

[*****100%*****] 3 of 3 completed

```
Out[36]:
```

	HighMinusLow	MovingAverageVolatility	StandardDeviation	Daily Return Percentage
Ticker				
China ETF	0.2629	4.816%	1.633%	-0.025%
Europe ETF	0.2324	3.378%	1.051%	0.055%
SP500	0.2806	2.985%	0.98%	0.058%

Key Takeaways:

- The China ETF appears to be the most volatile and risky investment among the three, with a negative average daily return during this period.
- The Europe ETF and the U.S. stocks show similar levels of volatility and positive returns, with the U.S. stocks having a slightly higher return.

Now, let's zoom out and look at the data since 2010, and we'll see the following:

```
In [37]: # Comparing Domestic vs. Foreign Stocks
start = datetime.date(2010, 8, 1)
end = datetime.date(2024, 8, 1)
investCompare(start, end, {"^GSPC": "SP500", "SPEU": "Europe ETF", "GXC": "China ETF"}
```

[*****100%*****] 3 of 3 completed

Out[37]:

	HighMinusLow	MovingAverageVolatility	StandardDeviation	Daily Return Percentage
Ticker				
China ETF	0.2629	4.521%	1.567%	0.006%
Europe ETF	0.2324	3.241%	1.233%	0.021%
SP500	0.2806	2.884%	1.085%	0.045%

Key Takeaways:

- Over this longer period, the U.S. equities appear to have provided the highest returns, followed by the Europe ETF and then the China ETF.
- The China ETF, while still showing higher volatility, has a positive average daily return over the long term, unlike the negative return observed in the shorter timeframe analysis.

While considering longer timeframes is generally recommended for investment evaluation, it's not a universal rule and there are valid arguments against solely relying on long-term performance.

Here are some points we could argue:

- Changing Market Dynamics: Long-term historical data might not accurately reflect current or future market conditions. Economic shifts, technological disruptions, heightened regulation, and geopolitical events can significantly impact investment performance, making past trends less relevant.
- Investment Goals and Time Horizon: An investor with a shorter time horizon (e.g., saving for a down payment) might prioritize short-term performance over long-term trends. Their investment strategy and risk tolerance would differ from someone with a longer-term goal (e.g., retirement).
- Missed Opportunities: Focusing solely on long-term trends might lead to missed opportunities in the short term. Active investors or traders might capitalize on short-term market fluctuations to generate returns, even if the long-term outlook is uncertain.
- Survivorship Bias: Long-term data often suffers from survivorship bias, meaning it only includes companies or investments that have survived over time. This can create a distorted view of historical performance, as failed investments are not included in the analysis.

It's important to consider both short-term and long-term factors when evaluating investment performance. A balanced approach that incorporates your individual goals, risk tolerance, and the current market environment is crucial for making informed investment decisions.

8. Conclusion

This lesson was a starting point for pulling in and analyzing price data in Python. We went into some specifics on different types of stocks and compared them using similar methods. We have also compared returns of different investment opportunities, and we used Python to illustrate different ways of calculating the volatility of investments. Keep these lessons in mind as we delve deeper into the connection between volatility and return in the next lesson. Up next, we'll expand this analysis by also taking into account the returns of different investments.

References

- Maginn, John L. *Managing Investment Portfolios Workbook: A Dynamic Process*. John Wiley & Sons, Inc., 2007.
- Standard & Poor's. *S&P U.S. Indices Methodology*. Nov. 2021, <https://www.spglobal.com/spdji/en/documents/methodologies/methodology-sp-us-indices.pdf>.
- Vanguard. "BLV Long-Term Bond ETF." <https://institutional.vanguard.com/investments/product-details/fund/0927>.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Bionic Turtle. "Three Approaches to Value at Risk (VaR) and Volatility (FRM T4-1)." *YouTube*, 14 Nov. 2018, <https://www.youtube.com/watch?v=qzOWm7PurHE>.

- **Required pages:** Whole video
- **Estimated reading time:** 18 minutes

2. Downey, Allen B. *Think Stats: Exploratory Data Analysis in Python*. Version 2.2.0, Green Tea Press, 2014. <https://greenteapress.com/thinkstats2/thinkstats2.pdf>.

- **Required pages:** Sections 5.2-5.4 (pp. 60–66)
- **Estimated reading time:** 15 minutes

Financial Asset & Return Analysis

Reading Time 60 minutes

Prior Knowledge Basic Python

Keywords Log and simple returns, Variance, Standard Deviation, Correlation, Covariance, Sharpe Ratio, Semivariance

This lesson will build upon the previous lessons' focus on price to finally expand the conversation to returns. We go much more in depth into how to calculate returns and how to compare two investments using various risk measures. Then, we take that a step further by using distributions to correctly model risk and returns.

```
In [1]: !pip install yfinance --upgrade --no-cache-dir
```

```

Requirement already satisfied: yfinance in /usr/local/lib/python3.11/site-packages
(0.2.54)
Collecting yfinance
  Downloading yfinance-0.2.65-py2.py3-none-any.whl (119 kB)
      119.4/119.4 kB 5.2 MB/s eta 0:00:00
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/site-packa
ges (from yfinance) (2.0.0)
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.11/site-packa
ges (from yfinance) (1.26.2)
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.11/site-pack
ages (from yfinance) (2.32.3)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.11/site
-packages (from yfinance) (0.0.11)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.11/site
-packages (from yfinance) (3.4.0)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.11/site-packag
es (from yfinance) (2023.3.post1)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.11/site-p
ackages (from yfinance) (2.4.6)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.11/site-pack
ages (from yfinance) (3.17.8)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.11/s
ite-packages (from yfinance) (4.12.2)
Collecting curl_cffi>=0.7
  Downloading curl_cffi-0.13.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.
whl (8.3 MB)
      8.3/8.3 MB 114.0 MB/s eta 0:00:00a 0:0
0:01
Collecting protobuf>=3.19.0
  Downloading protobuf-6.32.0-cp39-abi3-manylinux2014_x86_64.whl (322 kB)
      322.0/322.0 kB 443.6 MB/s eta 0:00:00
Collecting websockets>=13.0
  Downloading websockets-15.0.1-cp311-cp311-manylinux_2_5_x86_64.manylinux1_x86_64.m
anylinux_2_17_x86_64.manylinux2014_x86_64.whl (182 kB)
      182.3/182.3 kB 463.8 MB/s eta 0:00:00
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-packa
ges (from beautifulsoup4>=4.11.1->yfinance) (2.4.1)
Requirement already satisfied: cffi>=1.12.0 in /usr/local/lib/python3.11/site-packag
es (from curl_cffi>=0.7->yfinance) (1.15.1)
Collecting certifi>=2024.2.2
  Downloading certifi-2025.8.3-py3-none-any.whl (161 kB)
      161.2/161.2 kB 493.9 MB/s eta 0:00:00
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/s
ite-packages (from pandas>=1.3.0->yfinance) (2.8.2)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-pack
ages (from pandas>=1.3.0->yfinance) (2023.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.1
1/site-packages (from requests>=2.31->yfinance) (3.1.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packag
es (from requests>=2.31->yfinance) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/site-
packages (from requests>=2.31->yfinance) (1.26.15)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/site-packages
(from cffi>=1.12.0->curl_cffi>=0.7->yfinance) (2.21)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages
(from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.16.0)

```

```
Installing collected packages: websockets, protobuf, certifi, curl_cffi, yfinance
  Attempting uninstall: certifi
    Found existing installation: certifi 2022.12.7
    Uninstalling certifi-2022.12.7:
      Successfully uninstalled certifi-2022.12.7
  Attempting uninstall: yfinance
    Found existing installation: yfinance 0.2.54
    Uninstalling yfinance-0.2.54:
      Successfully uninstalled yfinance-0.2.54
Successfully installed certifi-2025.8.3 curl_cffi-0.13.0 protobuf-6.32.0 websockets-
15.0.1 yfinance-0.2.65
```

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

```
[notice] A new release of pip available: 22.3.1 -> 25.2
[notice] To update, run: pip install --upgrade pip
```

```
In [2]: import datetime
import numpy as np
import pandas as pd
import seaborn as sns
import yfinance as yfin

from matplotlib import pyplot as plt
from scipy import stats
```

1. Obtaining and Transforming Financial Data

1.1 Importing Data

Since this course has a more practical focus, we will start by pulling price data into Python and showing how to simply clean and transform the data so it's in a format that is easier to work with.

We use the `pandas_datareader` library in order to pull financial data. We will pull data from the FRED (Federal Reserve Economic Data) database. Keep in mind that we can use the `pandas_datareader` package to pull from many different sources, like OECD, Yahoo Finance, etc.

```
In [3]: # Starting and end dates
start = datetime.date(2019, 8, 1)
end = datetime.date(2024, 8, 1)

# Get data
df = yfin.download(["^GSPC", "^IXIC", "BTC-USD"], start, end, auto_adjust = False)

# Display the first five rows
df.head()
```

```
[*****100%*****] 3 of 3 completed
```

```
Out[3]:
```

	Ticker	BTC-USD	^GSPC	^IXIC
Date				
2019-08-01	10399.668945	2953.560059	8111.120117	
2019-08-02	10518.174805	2932.050049	8004.069824	
2019-08-03	10821.726562		NaN	NaN
2019-08-04	10970.184570		NaN	NaN
2019-08-05	11805.653320	2844.739990	7726.040039	

We've retrieved the data from the two most popular U.S. indices, the NASDAQ and S&P 500, along with the daily Bitcoin prices from the last five years. In this lesson, some basic data cleaning will be done by removing nulls (weekend data). Then, we'll have a DataFrame containing the dates and prices of three different assets. We need to compare returns instead of prices here for a couple of reasons:

1. Return is a scale-free summary of an investment opportunity.
2. Returns have statistical properties that are easier to work with. (This will be discussed more in this lesson.)

The next question is whether to use simple returns or log returns. Using the following variables, we can define the different types of return: p_{t-1} = final value, p_{t-0} = initial value.

Simple Returns Formula

$$R_{\text{simple}} = \frac{p_{t-1} - p_{t-0}}{p_{t-0}}$$

For example, if we were calculating yearly returns and on day 1, the portfolio was worth \$100 and at the end of the year it was worth \$125, the simple return would be $(125 - 100) / 100 = 0.25 = 25\%$ gain.

Log Returns Formula

$$R_{\text{log}} = \ln \left(\frac{p_{t-1}}{p_{t-0}} \right)$$

For example, if our portfolio was worth \$100 at the start of the year and \$80 by the end of the year, the log return would be $\ln(80/100) = -0.223 = -22.3\%$ loss.

Log returns are used in this case because it is a common assumption in many financial models that returns are normally distributed, and log returns have good mathematical properties, which make them easier to work with considering this assumption.

1.2 Calculate Log Returns, Remove Unused Columns, and Drop Nulls

We need to remove the nulls for the weekend dates. The following code snippet calculates the log returns for the S&P 500, NASDAQ, and Bitcoin and removes the original price columns.

```
In [4]: # Remove rows with missing values
df.dropna(inplace = True)

# Calculate log returns
df["SP500"] = np.log(df["^GSPC"]) - np.log(df["^GSPC"].shift(1))
df["NASDAQ"] = np.log(df["^IXIC"]) - np.log(df["^IXIC"].shift(1))
df["Bitcoin"] = np.log(df["BTC-USD"]) - np.log(df["BTC-USD"].shift(1))

# Remove original price columns
df.drop(["^GSPC",      "^IXIC", "BTC-USD"], axis = 'columns', inplace = True)

# Remove rows with missing values (again)
df.dropna(inplace = True)

# Display the first five rows
df.head()
```

```
Out[4]:
```

	Ticker	SP500	NASDAQ	Bitcoin
	Date			
	2019-08-02	-0.007309	-0.013286	0.011331
	2019-08-05	-0.030230	-0.035354	0.115474
	2019-08-06	0.012933	0.013784	-0.028132
	2019-08-07	0.000767	0.003767	0.039612
	2019-08-08	0.018588	0.022178	0.002044

1.3 Show Summary Stats for the Index Returns

This code snippet will generate descriptive statistics of your dataframe:

```
In [5]: df.describe()
```

Out[5]:

Ticker	SP500	NASDAQ	Bitcoin
count	1257.000000	1257.000000	1257.000000
mean	0.000498	0.000616	0.001453
std	0.013458	0.015952	0.041542
min	-0.127652	-0.131492	-0.464730
25%	-0.005256	-0.006802	-0.016203
50%	0.000866	0.001270	0.000365
75%	0.007240	0.009293	0.020095
max	0.089683	0.089347	0.191527

2. Variance and Standard Deviation

Investors have to keep volatility in mind as well when choosing an investment. For example, a pension fund may need to be extra careful with its money and will want to ensure they aren't getting into any extremely volatile investments. There are also hedge funds, which short stocks and even trade volatility with options. As you can see, whether you're risk-seeking or risk-averse, the volatility (risk) of an investment is something you should care about.

A simple measure of volatility is the variance. Variance is used to see how far away each data point in a set is away from the mean. Variance is calculated with the following steps:

- Take the difference between each data point and the mean
- Square each difference so that they're all positive values
- Sum up the squared results
- Divide this by the count of data points minus one

$$\sigma^2 = \frac{\sum(x_i - \overline{x})^2}{n-1}$$

where

σ^2 = sample variance x_i = value of one observation $\overline{x}$ = mean of all observations n = number of observations

The larger the variance, the further spread out it is from the mean. Variance treats all deviations from the mean the same way, regardless of whether they are less than or greater than the mean. A variance of zero would indicate that each data point is the same.

Standard deviation is easy to calculate once you have the variance. All you have to do is take the square root of the variance:

$$\sigma = \sqrt{\sigma^2}$$

where σ = standard deviation

Standard deviation is another commonly used statistical measure to quantify market volatility. You would expect newer growth stocks to have higher standard deviations and more established blue-chip stocks to have lower standard deviations of returns. We will illustrate this by comparing daily returns from the last five years of the S&P 500 and NASDAQ, as well as Bitcoin prices. While Bitcoin isn't necessarily a growth stock, it's still very new compared to the S&P 500 and NASDAQ, so you would expect to see bigger swings in the price when compared to those two stock indices.

This is illustrated below by taking the standard deviations of returns over the last five years. These results are as expected: both the S&P 500 and NASDAQ have very similar daily standard deviations. Bitcoin, on the other hand, has almost five times the standard deviation of the two stock indices.

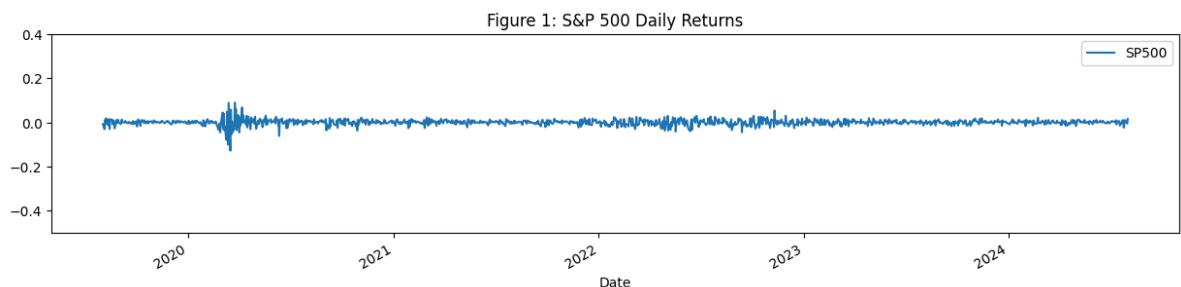
```
In [6]: df.std()
```

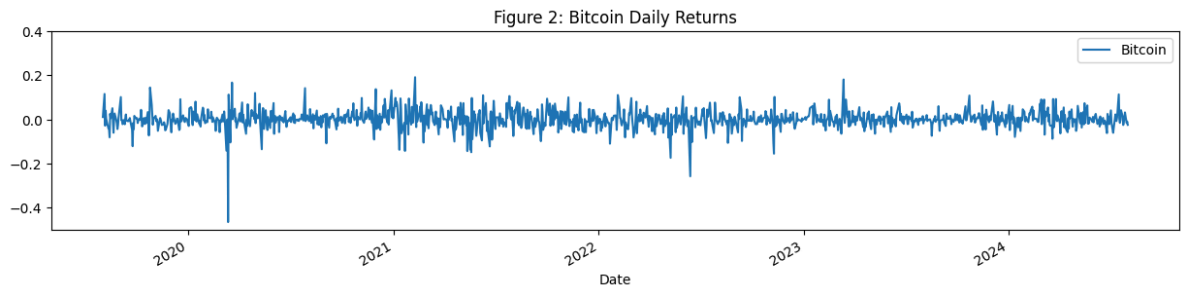
```
Out[6]: Ticker
SP500    0.013458
NASDAQ   0.015952
Bitcoin   0.041542
dtype: float64
```

We can also visualize this by graphing returns in Python using the plot method in pandas in conjunction with standardized y-axis limits so that we can ensure we're comparing returns on the same scale:

```
In [7]: # Generate two plots
ax1 = df.plot(figsize=(15, 3), y="SP500", title="Figure 1: S&P 500 Daily Returns")
ax2 = df.plot(figsize=(15, 3), y="Bitcoin", title="Figure 2: Bitcoin Daily Returns")

# Set y-axis limits for both plots
ax1.set_ylim(-0.5, 0.4)
ax2.set_ylim(-0.5, 0.4);
```





The above charts show clearly how much more volatile Bitcoin is compared to the S&P 500 and hence why the variance/standard deviation of returns is much higher.

Standard deviation is preferred over variance in most cases because variance is a squared result of the units of return. By taking the square root of this and thus obtaining standard deviation, the result is in the same unit as the underlying data, which in this case is returns. This intuitively makes it much easier to understand.

Keep in mind that a lower standard deviation is not necessarily preferable when considering investments. It all depends on the investor's risk preferences. A higher risk means a higher potential for rewards. Understanding the investor's perspective is key to determining what levels of risk an investor is comfortable with.

3. Covariance and Correlation

We are able to compare the performance of stocks that have different price levels by using returns and standard deviation. How can we look at the joint performance of two stocks? For this, we turn to covariance and correlation. We will start with covariance:

$$\frac{\sum (x_i - \overline{x})(y_i - \overline{y})}{N-1}$$

where

x_i = value of one observation of x y_i = value of one observation of y

$\overline{x}$ = mean of x $\overline{y}$ = mean of y N = number of observations

Covariance provides us some insight into how two variables move together. A positive covariance between stock returns would indicate that when one stock goes up, so does the other and vice versa. A negative covariance would mean that the two stocks move inversely, i.e., when one goes up, the other goes down.

Looking at the covariance matrix below for our three assets, we can see that these assets have a positive relationship with each other. It's hard to understand much more than that with these numbers, given that the units are not standardized.

3.1 Using a Covariance Matrix

While covariance is useful for determining the direction of two variables jointly, we can turn to correlation for a more standardized version of this. For now, when discussing correlation, we will use the Pearson's correlation coefficient. The other types of correlation will be discussed in a future lesson.

In [8]: `df.cov()`

Out[8]:

Ticker	SP500	NASDAQ	Bitcoin
Ticker			
SP500	0.000181	0.000203	0.000200
NASDAQ	0.000203	0.000254	0.000253
Bitcoin	0.000200	0.000253	0.001726

Here's how to interpret it:

- Diagonal values: These represent the variance of each asset. For example, 0.000149 is the variance of the S&P 500 daily log returns.
- Off-diagonal values: These represent the covariance between two different assets. For example, 0.000160 is the covariance between the daily log returns of the S&P 500 and NASDAQ.

All the values are positive, indicating a positive relationship between the returns of these assets. This means that when the return of one asset is positive, the return of the other assets tends to be positive as well.

3.2 Using the Pearson Correlation Coefficient Formula

The Pearson correlation coefficient is a measure of the strength of a linear relationship between two variables. The formula is as follows:

$$\rho_{\{X,Y\}} = \frac{\text{cov}(X,Y)}{\sigma_{\{X\}} \sigma_{\{Y\}}}$$

where

cov = Covariance $\sigma_{\{X\}}$ = Standard Deviation of X $\sigma_{\{Y\}}$ = Standard Deviation of Y

Correlation is used in statistics to quantify the degree to which two variables move in a linear relation to each other. Correlation can range from -1 to 1 inclusive. A correlation of 1 means perfect correlation; the variables move exactly in tandem with one another. Perfect tandem for positive correlation means that if we know variable X increases, then variable Y also increases. A correlation of -1 indicates perfect inverse correlation. This is also perfect tandem, but in the opposite direction. Here, if we know variable X increases, then variable Y decreases. A correlation of 0 indicates there is no linear distinguishable relationship between

two variables; therefore, it would be impossible to make predictions of one variable given the other. In this case, if we know variable X increases, then variable Y is equally likely to increase or decrease.

A benefit of correlation over covariance is that correlation is capped from -1 to 1 while covariance can be from -inf to inf. This makes covariance a harder statistic to understand intuitively. Correlation is also proportional, which will be shown in the video below.

Keep in mind that a lot of models and financial concepts assume a constant correlation, but this is rarely the case. Correlation changes over time and will likely even change if you adjust the time range from which you're measuring correlation.

When comparing the correlations of the assets we've been using so far, you can see it paints a clearer picture than the covariance did.

```
In [9]: round(df.corr(), 3)
```

```
Out[9]:
```

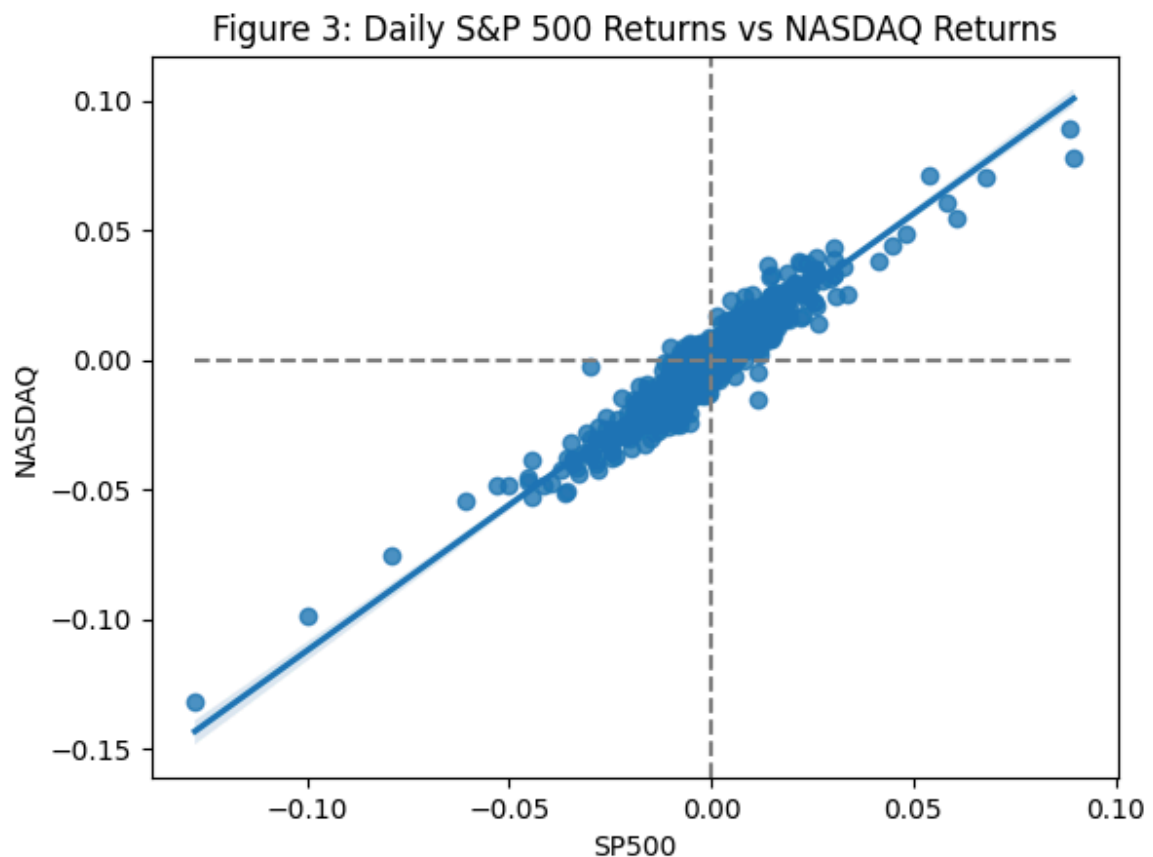
Ticker	SP500	NASDAQ	Bitcoin
Ticker			
SP500	1.000	0.947	0.358
NASDAQ	0.947	1.000	0.381
Bitcoin	0.358	0.381	1.000

All of the variables have a positive relationship, which we saw with the covariance matrix previously. Here, we can also see the strength of the relationships: the S&P 500 is strongly correlated with NASDAQ since we've obtained a 0.943 Pearson's correlation coefficient. The relationship between NASDAQ and Bitcoin is still positive but much weaker with a 0.205 Pearson's correlation coefficient.

When we chart the returns between these variables, we can see evidence of the correlations above, visually:

```
In [10]: # Create scatter plot with regression line
chart = sns.regplot(x="SP500", y="NASDAQ", data=df).set(
    title="Figure 3: Daily S&P 500 Returns vs NASDAQ Returns"
)

# Add vertical line at x=0 and horizontal line at y=0
plt.axvline(0, 0, 1, dash_capstyle="butt", linestyle="--", color="grey")
plt.plot([min(df.SP500), max(df.SP500)], [0, 0], linestyle="--", color="grey");
```

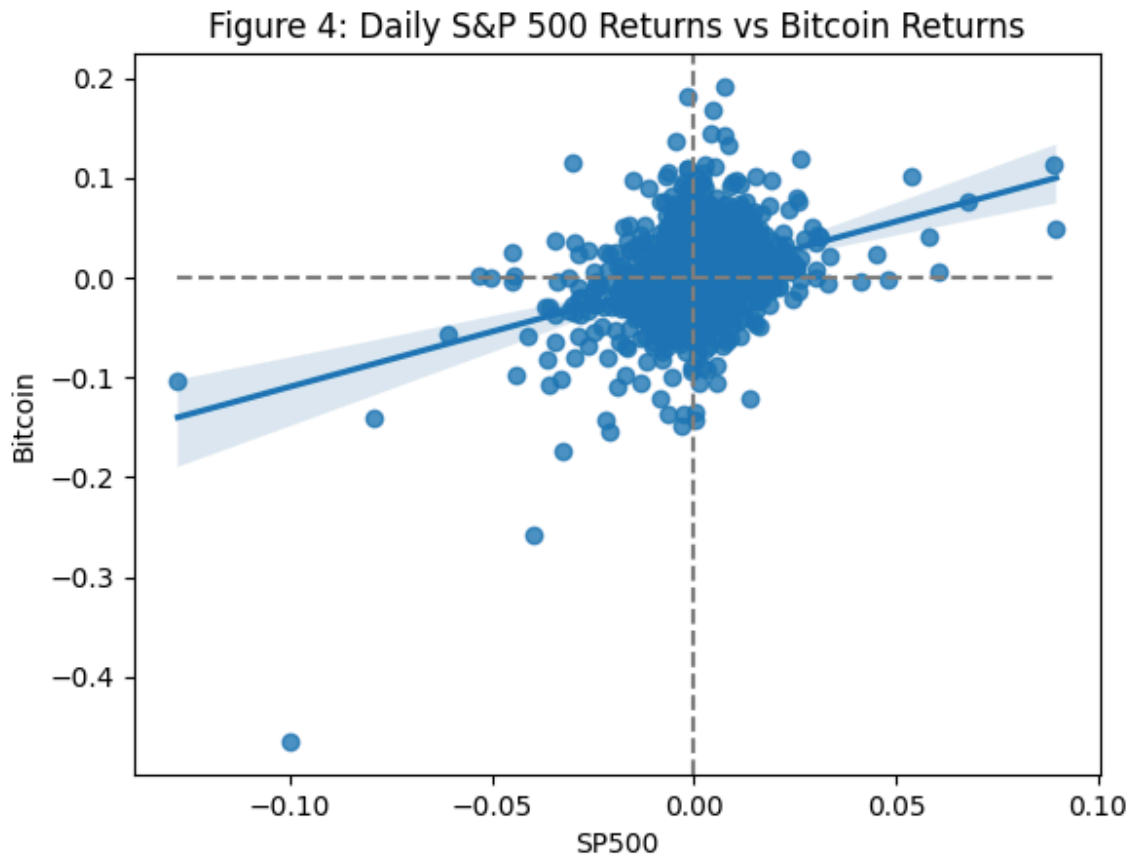


This plot visually shows the relationship between the daily log returns of the S&P 500 and NASDAQ, including a regression line that represents the best-fit linear relationship between the two variables. The dashed lines at $x=0$ and $y=0$ help to visualize the positive and negative returns for each asset. This relationship shows nearly perfect correlation.

Now let's visualize relationship between S&P 500 and Bitcoin daily returns.

```
In [11]: # Create scatter plot with regression line
sns.regplot(x="SP500", y="Bitcoin", data=df).set(
    title="Figure 4: Daily S&P 500 Returns vs Bitcoin Returns"
)

# Add vertical line at x=0 and horizontal line at y=0
plt.axvline(0, 0, 1, dash_capstyle="butt", linestyle="--", color="grey")
plt.plot([min(df.SP500), max(df.SP500)], [0, 0], linestyle="--", color="grey");
```



The relationship here is much more scattered even though the relationship is still slightly positive. These were charted using the seaborn `regplot()` method, which also shows confidence intervals around the regression line, as seen above.

3.3 The Sharpe Ratio

Are there any statistics we can use to quantify not just return but also risk? For this, we turn to the Sharpe ratio.

The Sharpe ratio allows an investor to understand the relationship between the return of an investment and its volatility. The formula is as follows:

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p}$$

where

R_p = Return of the portfolio R_f = Risk-free rate σ_p = Standard deviation of the portfolio

In many cases with interest rates so low, investors will assume the risk-free rate to be 0, making the ratio:

$$\text{Sharpe Ratio} = \frac{R_p}{\sigma_p}$$

Notice anything about the denominator? Yes, that's right: we use the standard deviation here to represent risk.

This measure is used as a way of scaling the return of an investment depending on how much risk is taken. In other words, the higher the standard deviation, the more the risk-weighted return is reduced. Let's use the same example as above with S&P 500 and Bitcoin daily returns over the last five years.

```
In [12]: # Calculates the Sharpe Ratios for both the S&P 500 and Bitcoin
Sharpe_Ratio_SP500 = df["SP500"].mean() / df["SP500"].std()
Sharpe_Ratio_Bitcoin = df["Bitcoin"].mean() / df["Bitcoin"].std()

# Print the results
print("Sharpe Ratio of S&P 500: ", round(Sharpe_Ratio_SP500, 5))
print("Sharpe Ratio of Bitcoin: ", round(Sharpe_Ratio_Bitcoin, 5))
```

```
Sharpe Ratio of S&P 500: 0.03699
Sharpe Ratio of Bitcoin: 0.03498
```

Based on these results, the S&P 500 has a slightly higher Sharpe Ratio (0.03699) compared to Bitcoin (0.03498). This suggests that the S&P 500 has provided a slightly better risk-adjusted return over the analyzed period.

One major flaw with the Sharpe ratio is that it uses the standard deviation of returns in the denominator, which assumes that returns are normally distributed. This may not always--and is actually rarely--the case. This will be explored further in a future lesson.

4. Semivariance

How can we refine the Sharpe ratio to give an even better measure of risk-adjusted returns? Semivariance is the answer. Semivariance, also known as downside risk, is a more refined version of a standard deviation. Standard deviation looks at both the upside and downside risk of an investment. Most investors, unless you're trading short, care much more about the downside risk than the upside risk. In other words, if you bought a stock and are looking at the Sharpe ratio, you wouldn't want this number to be penalized for how far it moves to the upside. Most of the time, an investor will be much more concerned with the downside risk of a stock.

$$\text{Semivariance} = \frac{1}{n} \sum_{r_i < \overline{r}} (r_i - \overline{r})^2$$

where

r_i = value of one observation
 $\overline{r}$ = mean of all observations
 n = number of observations

Semivariance can be thought of as an estimator of variance of the returns that are less than their average. This can be used to estimate the average loss a portfolio could incur,

assuming normal distributions of returns.

Conversely, if we are short a security, we could still use semivariance, but this time, it would focus on the returns that are positive. If we are short, drops in the price create upside, so we would not want to penalize this volatility in our Sharpe ratio. However, if there are large deviations in the upward direction, then this will contribute to semivariance. In short, semivariance uses either the positive or negative returns.

Now let's compute semivariance for both the S&P 500 and Bitcoin:

```
In [13]: # Calculate the mean return for each
sp500mean = df["SP500"].mean()
BTCmean = df["Bitcoin"].mean()

# Calculate semivariance for each
sp500_semivariance = ((df[df["SP500"] < sp500mean]["SP500"] - sp500mean) ** 2).mean()
BTC_semivariance = ((df[df["Bitcoin"] < BTCmean]["Bitcoin"] - BTCmean) ** 2).mean()

# Print the semivariance results
print("Semivariance of S&P 500: ", round(sp500_semivariance, 5))
print("Semivariance of Bitcoin: ", round(BTC_semivariance, 5))
```

Semivariance of S&P 500: 0.00021

Semivariance of Bitcoin: 0.00181

This output shows that Bitcoin has a much higher semivariance (0.00181) compared to the S&P 500 (0.00021). This indicates that Bitcoin has experienced significantly larger negative deviations from its average return, suggesting higher downside risk.

5. How Are Stock Returns Distributed?

Many models and theories surrounding stocks assume a normal distribution. We will try to determine that here with a data-based analysis. Properties of a Gaussian distribution are as follows:

- Mean, median, and mode are all the same.
- The data is symmetrical, meaning there are equal counts of observations on both sides of the mean.
- In normally distributed data, 68.25% of all cases fall within +/- one standard deviation from the mean, 95% of all cases fall within +/- two standard deviations from the mean, and 99.7% of all cases fall within +/- three standard deviations from the mean.

Let's start by pulling 20 years of daily price data for the S&P 500. We'll use similar methods we've used in the last few lessons to pull this data and will calculate the log returns here.

One quick way of doing this is to determine how many data points we have on either side of the mean here. We have a bit more than 5,000 data points. The below code takes the count

of data points greater than the mean and divides it by the total number of data points. This will give us the percentage of data points greater than the mean.

```
In [14]: # Starting and end dates
start = datetime.date(2004, 8, 1)
end = datetime.date(2024, 8, 1)

# Get the data
prices = pd.DataFrame(yfin.download(["^GSPC"], start, end, auto_adjust = False)["Ad

# Rename column to make names more intuitive
prices = prices.rename(columns={"^GSPC": "SP500"})
df = np.log(prices) - np.log(prices.shift(1))
df = df.iloc[1:, 0:]

[*****100%*****] 1 of 1 completed
```

5.1 Are Returns Symmetric?

One quick way of doing this is to determine how many data points we have on either side of the mean here. We have a bit more than 5,000 data points here. The below code takes the count of data points greater than the mean and divides it by the total number of data points. This will give us the percentage of data points greater than the mean.

```
In [15]: (len(df[df.SP500 > df.SP500.mean()])) / (len(df))

Out[15]: 0.5235446056030201
```

The output will be a number between 0 and 1, representing the proportion of S&P 500 daily returns that are greater than the mean return. This can give you an idea of the skewness of the return distribution. A value close to 0.5 suggests a roughly symmetrical distribution, while a value significantly greater than 0.5 suggests a negative skew (more returns below the mean).

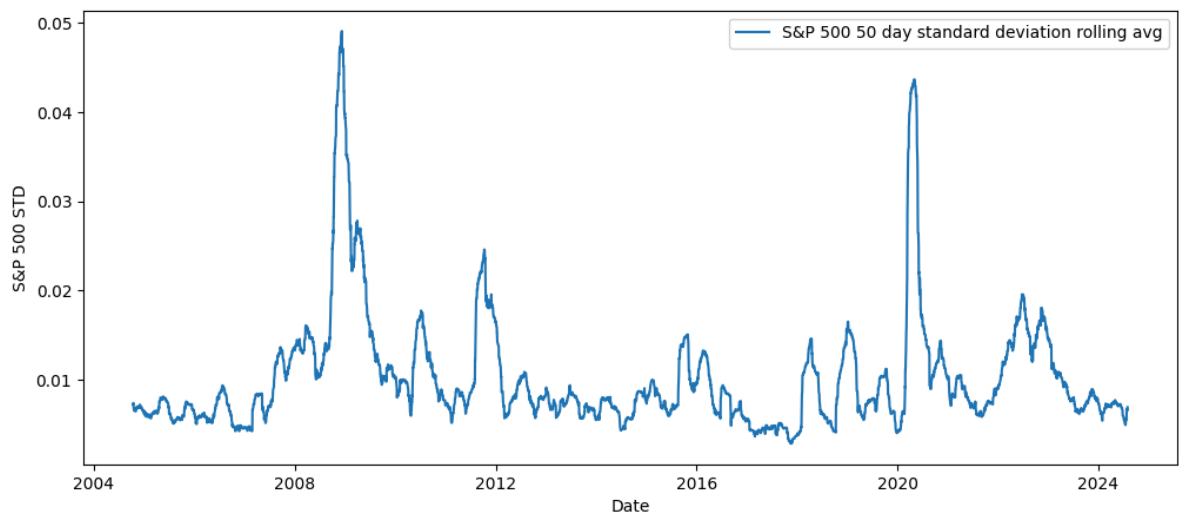
We're getting about 52.6% of data points being greater than the mean, which shows we have a slightly negative skew to this dataset. We can't rule out symmetric returns based on this since it is only a sample of data and is reasonably close to the 50% mark. This makes it hard to say for certain whether S&P 500 returns are symmetric or not, but it is still a reasonable assumption to make here.

5.2 Is Volatility Constant?

The following code calculates and plots the rolling 50-day standard deviation of the S&P 500 log returns, providing a visualization of the volatility over time.

```
In [16]: # Calculate rolling standard deviation
vols = pd.DataFrame(df.SP500.rolling(50).std()).rename(columns={"SP500": "S&P 500 S
```

```
# set figure size and plot rolling standard deviations
plt.figure(figsize=(12, 5))
sns.lineplot(
    x="Date",
    y="S&P 500 STD",
    data=vols,
    label="S&P 500 50 day standard deviation rolling avg",
)
plt.show()
```



From the above plot, it can be clearly seen that volatility is anything but constant. This adds another layer of complexity to modeling stock returns, especially in the many models which assume constant volatility.

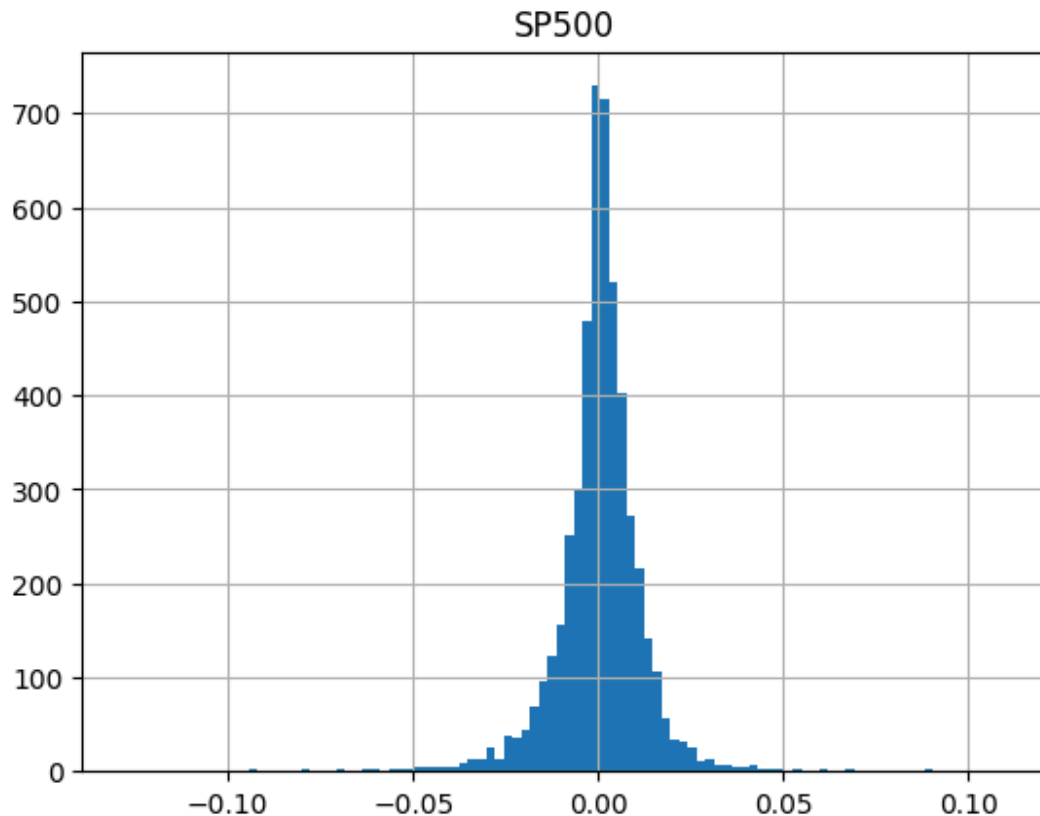
6. Are Stock Returns Normally Distributed?

The normal distribution is one of the most common distributions used in modeling random variables. Indeed, many phenomena in the natural and social sciences can be modeled by normal distribution. One of the great advantages of the normal distribution is its simplicity. We can completely describe a normal distribution through two numbers: one for the center of the distribution and one for the uncertainty about that center. The first number refers to the mean, and the second number refers to the standard deviation.

Once we have these two numbers, we can draw inferences, estimate percentiles, compute probabilities that a point falls within a region, and more. If our data is well represented by the normal distribution, then we can confidently use the mean and standard deviation to report our portfolio expected returns and volatilities. If our data is not well represented by the normal distribution, then we need to find other distributions that are more suitable. Thus, when we have a distribution of stock returns, for example, we'll want to start this assessment by visualizing the returns to see if they appear to be normal. Of course, we can follow this up with more quantitative assessments by running statistical tests.

We can visualize the data using the `hist()` method. We pass in `bins = 100` as a parameter to determine the amount of buckets to place the data in. The more bins you have, the more granular the data will look in a histogram. Increasing the bins too much may result in slightly noisy data, which will make it tougher to determine a normal distribution.

```
In [17]: df.hist(bins=100);
```



The chart above looks like it could be normally distributed, but we need to be a little more scientific to determine if that's actually the case or not.

6.1 Conducting a Normality Test

We can use the `normaltest()` method to determine if the sample data could fit a normal distribution. This method uses D'Agostino and Pearson's normality test, which combines skew and kurtosis to produce an omnibus test of normality.

```
In [18]: stats.normaltest((np.array(df.SP500)))
```

```
Out[18]: NormaltestResult(statistic=1234.7310969195069, pvalue=7.61288004534643e-269)
```

The output of this code includes two values:

- **Statistic:** A test statistic that measures the deviation from normality. Higher values indicate a greater deviation.

- p-value: The probability of observing the data if it were truly normally distributed. A small p-value (typically less than 0.05) suggests that the data is not normally distributed.

The output above shows the results of the D'Agostino and Pearson normality test on our S&P 500 log return data. The statistic of 1234.73 is very high, indicating a substantial deviation from a normal distribution. The extremely small p-value (7.61e-269) provides strong evidence to reject the null hypothesis that the data is normally distributed. In simpler terms, this test strongly suggests that the S&P 500 daily log returns in our dataset do not follow a normal distribution.

6.2 Testing Skewness and Kurtosis

As one added testing step, we can test the skewness and kurtosis of our distribution using the Jarque-Bera test. The test statistic will always be greater than zero. The further the test statistic is from zero, the more likely the sample data does not match a normal distribution.

Lucky for us, Python has another library for us to use here, which really simplifies the analysis. From the `scipy.stats` library, we can apply the `jarque_bera()` method directly to our data to get the test statistic.

```
In [19]: stats.jarque_bera((np.array(df.SP500))).pvalue
```

```
Out[19]: 0.0
```

A small p-value (typically less than 0.05) indicates that the data is likely not normally distributed. This output indicates that the p-value from the Jarque-Bera test is 0.0 (or extremely close to zero). This strongly suggests that the S&P 500 daily log returns in your dataset are not normally distributed.

6.3 Where Does Our Gaussian Distribution Break Down?

According to the normality test, our data is not normally distributed despite the histogram looking like it may be. So why does the data fail the normality test? The answer likely comes down to fat tails. Fat tails essentially mean that extreme events (± 3 standard deviations away from the mean) are more likely than the normal distribution would imply.

To determine how many standard deviations away from the mean a specific number is, we need to use

$$\frac{X - \bar{X}}{\text{Sample standard deviation}}$$

Let's do this for the min and max of the sample data:

```
In [20]: # minimum and maximum daily log returns
dfMax = df.SP500.max()
dfMin = df.SP500.min()
```

```

# Print maximum and minimum daily log returns
print("Maximum return of sample data is: ", round(dfMax, 5))
print("Minimum return of sample data is: ", round(dfMin, 5))
print(' - - - - -')

# Calculates the number of standard deviations from the mean return
num_dev_max = (df.SP500.max() - df.SP500.mean()) / df.SP500.std()
num_dev_min = (df.SP500.min() - df.SP500.mean()) / df.SP500.std()

# Print num_dev_max and num_dev_min
print("Number of standard deviations from the mean for the maximum return: ", round
print("Number of standard deviations from the mean for the minimum return: ", round

```

Maximum return of sample data is: 0.10957

Minimum return of sample data is: -0.12765

- - - - -

Number of standard deviations from the mean for the maximum return: 9.0497

Number of standard deviations from the mean for the minimum return: -10.60025

This output provides valuable insights into the distribution of S&P 500 daily log returns, particularly highlighting the presence of outliers or fat tails:

- Maximum and Minimum Returns: The maximum daily return of approximately 10.957% and the minimum daily return of approximately -12.765% show the range of returns observed in our data.
- Z-scores: The z-scores for the maximum and minimum returns are 9.05 and -10.6, respectively. These are extremely high z-scores, indicating that both the maximum and minimum returns are significant outliers, far away from the mean in terms of standard deviations.

In a normal distribution, you would rarely observe data points more than 3 standard deviations away from the mean. These high z-scores suggest that the S&P 500 daily returns have fatter tails than a normal distribution, meaning extreme events (large positive or negative returns) are more likely than would be expected under normality.

This information is crucial for risk management and modeling, as relying on the assumption of normality can lead to underestimating the probability of extreme events and potential losses.

These standard deviations are humongous when compared to the normal distribution. We can see this analytically when we plug in the z score to the `norm.cdf()` method to determine the probability this value could be in a normal distribution:

```
In [21]: stats.norm.cdf(-10.60)
```

```
Out[21]: 1.4899011272964664e-26
```

This implies that the chance we could have a move as small as -12.77% is 1.49e-26. This probability is so low that we would never expect an event like this to happen in our lifetime.

We have multiple events like this, as illustrated by the minimum and maximum.

Going further with this idea, based on normal distribution z tables, we would expect 99.7% of our data points to be within ± 3 standard deviations from the mean. Let's determine this for our sample data. First off, we need to find the cut-off values at ± 3 standard deviations:

```
In [22]: # Calculates the upper and lowers bounds
upper = (3 * df.SP500.std()) + df.SP500.mean()
lower = (-3 * df.SP500.std()) + df.SP500.mean()

# Print the results
print("Upper bound: ", round(upper, 5))
print("Lower bound: ", round(lower, 5))
```

```
Upper bound:  0.03654
Lower bound: -0.0359
```

The above two calculations would imply that 99.7% of all of our data points should be in between -0.0359 and 0.03654.

Since we have 5,031 data points, we would expect about 15 (i.e., 3% of 5,031) of them to be outside of that range if normality was held. Now let's see how many we actually have. Let's filter the `df` DataFrame to select rows where the S&P 500 daily log returns are outside the range of upper bound and lower bound. Then, we'll count the number of the data points that fall in this range:

```
In [23]: # Calculates the number of data points
len(df[(df["SP500"] < lower) | (df["SP500"] > upper)])
```

```
Out[23]: 84
```

That's a significant number of outliers considering that in a normal distribution, you would expect only about 0.3% of the data points to fall outside the range defined by 3 standard deviations from the mean.

This finding further supports the conclusion from the normality tests that the S&P 500 daily log returns are not normally distributed and exhibit fat tails. The presence of these outliers highlights the importance of considering alternative distributions and risk measures when modeling and analyzing financial data.

7. Non-Gaussian Distributions

One potential alternative distribution we could use to forecast stock returns is the Student's t-distribution. This is very similar to a normal distribution except it has heavier tails.

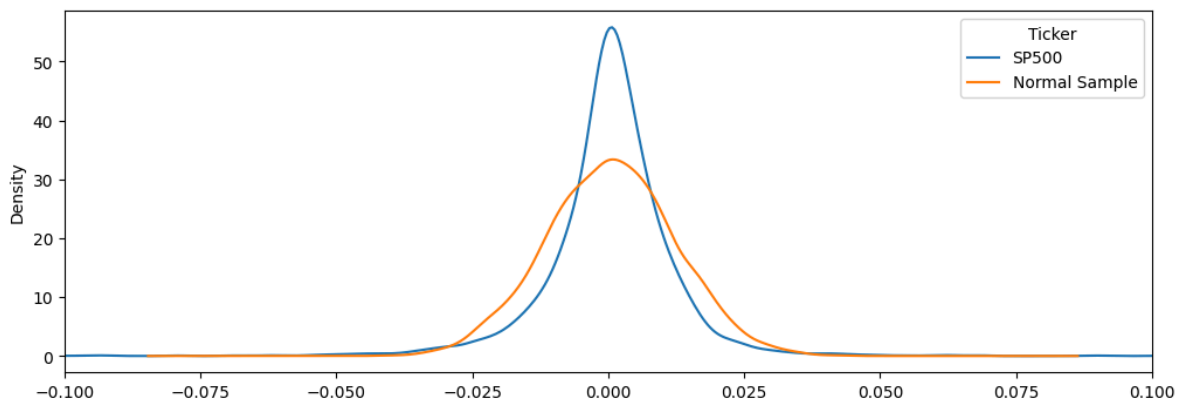
Theoretically, this sounds perfect for daily returns based on what we've seen up to this point.

Let's proceed with a visual inspection of the distribution of our data by superimposing the normal distribution on the kernel density estimation (KDE) of S&P 500 returns:

```
In [24]: # Sampling from normal distribution
np.random.seed(222)
normal_dist = stats.norm.rvs(size=len(df["SP500"]), loc = df["SP500"].mean(), scale

# Creating an additional column in df in order to use the KDE plot functionality of
df['Normal Sample'] = normal_dist

# Plotting the KDE plots
df[['SP500', 'Normal Sample']].plot(kind = 'kde', xlim = (-0.1, 0.1), figsize = (12
```



The S&P 500 returns seem a lot more leptokurtic. A leptokurtic distribution has fatter tails and a sharper peak than a normal distribution. By definition, leptokurtic distributions have kurtosis greater than 3.

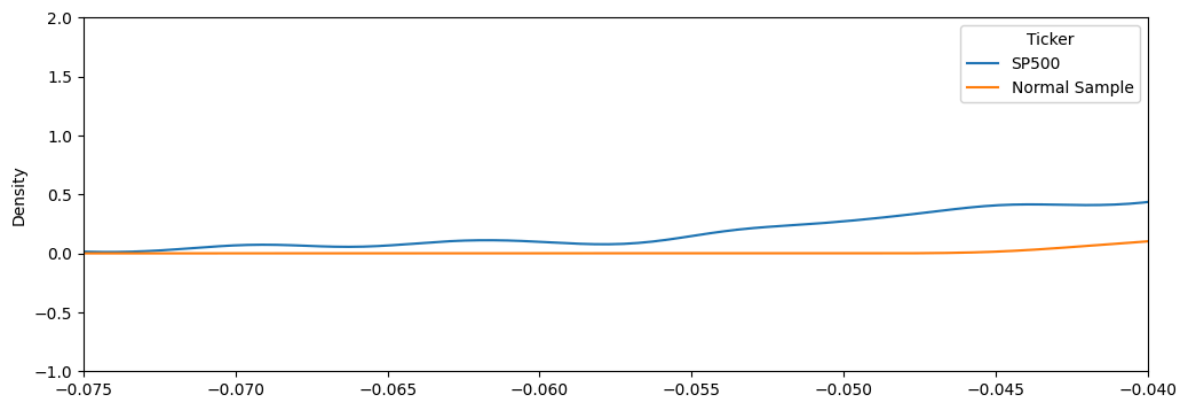
```
In [25]: df.SP500.kurt()
```

```
Out[25]: 13.180839169084916
```

Indeed, the kurtosis of the S&P 500 is significantly greater than 3, confirming that the distribution of S&P 500 daily log returns is highly leptokurtic.

Now, let's check if the tails of the S&P 500 are also fatter than those of a normal distribution. The following code creates a kernel density estimation (KDE) plot to compare the tails of the actual S&P 500 daily log returns with the tails of the generated normal sample. It focuses on the left tail (negative returns) by setting the x-axis limits to (-0.075, -0.04).

```
In [26]: # Observing the tails
df[['SP500', 'Normal Sample']].plot(kind = 'kde', xlim = (-0.075, -0.04), ylim = (-
```

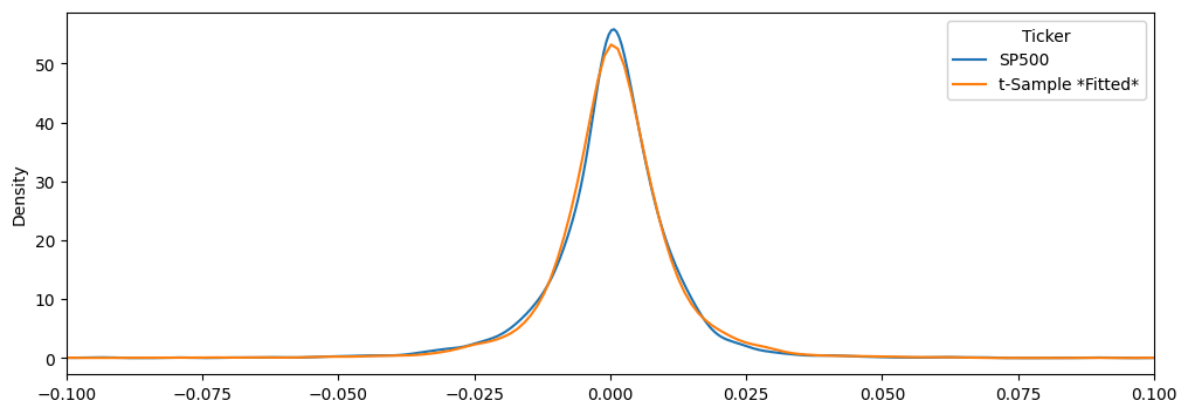


The resulting plot visually shows the differences in the tails of the two distributions. We observe that the actual S&P 500 returns have a fatter left tail than the normal distribution, indicating a higher probability of large negative returns. This visualization provides further evidence of the non-normality of the S&P 500 returns and the presence of fat tails.

At this stage, we will calibrate the parameters of the Student's t-distribution using Maximum Likelihood Estimation (MLE) to align the distribution closely with our observed data. The following code fits a Student's t-distribution to the S&P 500 daily log returns using [Maximum Likelihood Estimation \(MLE\)](#) and then creates a KDE plot to compare the fitted t-distribution with the actual data.

```
In [27]: # Fit the t-distribution using MLE
params = stats.t.fit(df.SP500)

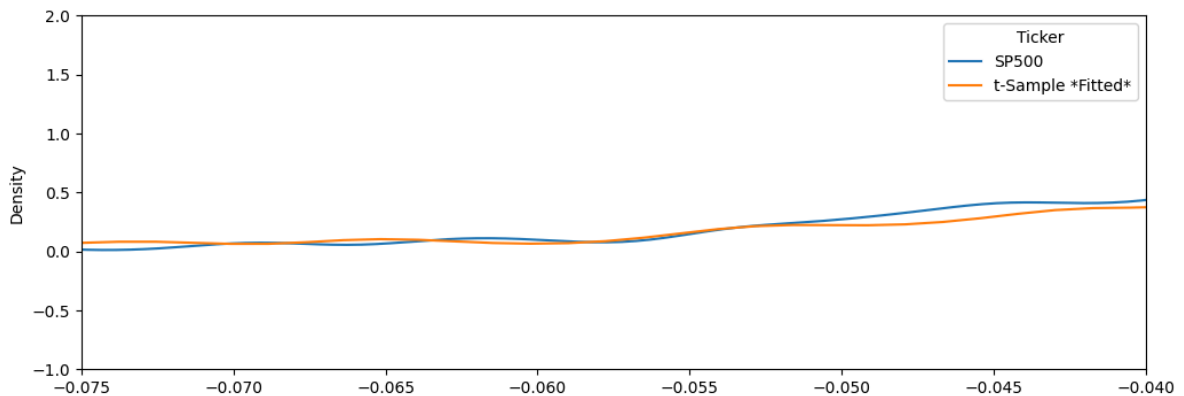
# We plot the fitted distribution against the kde of the data
df['t-Sample *Fitted*'] = stats.t.rvs(*params, size = len(df))
df[['SP500', 't-Sample *Fitted*']].plot(kind = 'kde', figsize = (12,4), xlim = (-0.
```



From the visual inspection, it's evident that the synthetic data generated from the fitted Student's t-distribution offers a more accurate approximation of our actual data compared to what we get from a normal distribution.

Let's check the tails once more. We will again zoom in to observe proximity between the left tail of the actual S&P 500 daily log returns with the left tail of the fitted Student's t-distribution:

```
In [28]: # Plot left tail area
df[['SP500', 't-Sample *Fitted*']].plot(kind = 'kde', figsize = (12,4), xlim = (-0.
```



This time, the tails seem to be better explained by the t-distribution. Here, we can observe that the fitted t-distribution captures the fat tail of the S&P 500 returns more accurately than the normal distribution. This further demonstrates the suitability of the t-distribution for modeling financial data that exhibits fat tails and the potential for extreme events.

8. Conclusion

This lesson covered surface-level returns and volatility metrics to compare stocks. We also discussed downside risk measures and ways to jointly compare different financial assets. Then, we delved into more advanced metrics modeled with different statistical distributions. We focused for the first time on comparing daily stock return data to normal distributions. We also touched on other potential distributions we could use to model this data.

References

- D'Agostino, Ralph B. "An Omnibus Test of Normality for Moderate and Large Size Samples." *Biometrika*, vol. 58, no. 2, 1971, pp. 341–348, <https://doi.org/10.1093/biomet/58.2.341>.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Hiranabe, Kenji. "Matrix World." *Linear Algebra for Everyone*. March 2023, <https://github.com/kenjihiranabe/The-Art-of-Linear-Algebra/blob/main/MatrixWorld.pdf>.

- **Required pages:** Pages 2–4
- **Estimated reading time:** 20 minutes

Special Matrices for Equity Analysis

Reading Time 60 minutes

Prior Knowledge Basic Matrix Operation, Linear Algebra, Basic Python

Keywords Stock return correlation, Symmetric matrices, Matrix diagonalization, Triangular matrices, Positive Definite Matrices, Cholesky Factorization

In this lesson, we are going to introduce different types of matrices. These matrices play important roles in financial analysis. With sound understanding of the characteristics and operation of these matrices, you will have a good start to learn many financial analytical techniques. During the lesson, we will also use stock prices and returns as examples to demonstrate the usage of these matrices. We will use Apple, Ford Motor, and Walmart's stock returns to explain the applications of these matrices.

1. Symmetric Matrices

1.1. Definition of Symmetric Matrices

A **symmetric matrix** is a square matrix whose transpose is the same as the matrix itself. A square matrix is a matrix that has the same number of rows as the number of columns. The following example demonstrates the concept of a symmetric matrix.

$$A = \begin{bmatrix} m & a & b \\ a & n & c \\ b & c & p \end{bmatrix}$$

$$A^T = \begin{bmatrix} m & a & b \\ a & n & c \\ b & c & p \end{bmatrix}$$

In the above example, the second matrix is the transpose of A matrix. The elements of the original matrix and its transpose are the same. In mathematical notation, for matrix A , if $A = A^T$, then A is symmetric. To be more specific, the element at position (i,j) is equal to the element at position (j,i) for all i and j .

Here are some key properties of a symmetric matrix:

1. The eigenvalues of the matrix are real numbers.

Loading [MathJax]/extensions/Safe.js
vectors of the matrix are orthogonal to each other.

3. The matrix is diagonalizable.
4. The rank of the matrix is equal to the number of non-zero eigenvalues.

When two vectors are **orthogonal**, it means that the vectors are perpendicular to each other. It also means the inner product of the two vectors is 0. These two vectors are linearly independent. Next, we will talk about the diagonalizability of a symmetric matrix.

1.2. Diagonalization of Symmetric Matrices Matrix A is diagonalizable if there exists a diagonal matrix Λ such that

$$A = B \Lambda B^{-1}$$

If A is a symmetric matrix, the values on the main diagonal of Λ are eigenvalues of A . B is a matrix whose columns are eigenvectors of A .

Let's look at one example of diagonalization of a symmetric matrix.

We have a symmetric matrix M as follows:

$$M = \begin{bmatrix} 6 & -2 & -1 \\ -2 & 6 & -1 \\ -1 & -1 & 5 \end{bmatrix}$$

In order to obtain eigenvalues, we need to set the characteristic polynomial to 0, or solve for the characteristic equation as follows:

$$\det(M - \lambda I) = \det \begin{bmatrix} 6-\lambda & -2 & -1 \\ -2 & 6-\lambda & -1 \\ -1 & -1 & 5-\lambda \end{bmatrix}$$

$$\Downarrow$$

$$(6-\lambda)(6-\lambda)(5-\lambda) + (-2)(-1)(-1) + (-2)(-1)(-1)(-1)(6-\lambda) - (-2)(-2)(5-\lambda) - (-1)(-1)(6-\lambda) = 0$$

$$\Downarrow$$

$$-\lambda^3 + 17\lambda^2 - 90\lambda + 144 = -(\lambda - 8)(\lambda - 6)(\lambda - 3) = 0$$

Hence, the eigenvalues are $\lambda_1 = 8$, $\lambda_2 = 6$ and $\lambda_3 = 3$. The corresponding eigenvectors are as follows:

$$\nu_1 = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}, \nu_2 = \begin{bmatrix} -1 \\ -1 \\ 2 \end{bmatrix}, \nu_3 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

One thing to pay attention to is that the inner products of each eigenvector pairs are all 0 ($\nu_1^T \nu_2 = 0, \nu_2^T \nu_3 = 0, \nu_1^T \nu_3 = 0$). This means that they are orthogonal to each other.

We can further normalize eigenvectors to be unit vectors as follows:

Loading [MathJax]/extensions/Safe.js

$$\begin{bmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix},$$

$$\omega_2 = \begin{bmatrix} -\frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \end{bmatrix}, \omega_3 = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{bmatrix}$$

Now we can write down the diagonalized symmetric matrix M .

$$M = B \Lambda B^{-1}$$

where $\Lambda = \begin{bmatrix} 8 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

$$B = \begin{bmatrix} -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{bmatrix}$$

Because B is an orthogonal matrix, $B^T = B^{-1}$. We can also write the diagonalized symmetric matrix as follows:

$$M = B \Lambda B^T$$

Transforming a symmetric matrix to the diagonalized form will make the computation of matrices more efficient. It also helps decompose a matrix into simpler forms for analysis. Diagonalization of a matrix is also an essential tool to help solve the linear transformation issue.

Now we have learned many properties of symmetric matrices. Let's take a look at some common symmetric matrices, like the covariance matrix and the correlation matrix. We'll look at the covariance matrix first.

1.3. Covariance Matrix

We introduced the covariance matrix in the previous lesson. A covariance matrix is a square matrix giving the variances of the variables on the diagonal of the matrix and the covariances between each pair of the variables on the off-diagonal. Based on the structure of a covariance matrix, it is a symmetric matrix. Let's calculate the covariance matrix for stock returns for Apple, Ford Motor, and Walmart as an example.

```
In [1]: !pip install yfinance --upgrade --no-cache-dir
```

Requirement already satisfied: yfinance in /usr/local/lib/python3.11/site-packages (0.2.54)

Collecting yfinance

Downloading yfinance-0.2.65-py2.py3-none-any.whl (119 kB)

119.4/119.4 kB 4.2 MB/s eta 0:00:00

Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.0.0)

Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.11/site-packages (from yfinance) (1.26.2)

Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.32.3)

Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.11/site-packages (from yfinance) (0.0.11)

Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (3.4.0)

Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.11/site-packages (from yfinance) (2023.3.post1)

Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.4.6)

Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.11/site-packages (from yfinance) (3.17.8)

Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.11/site-packages (from yfinance) (4.12.2)

Collecting curl_cffi>=0.7

Downloading curl_cffi-0.13.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (8.3 MB)

8.3/8.3 MB 104.7 MB/s eta 0:00:00 0:0

0:01

Collecting protobuf>=3.19.0

Downloading protobuf-6.32.0-cp39-abi3-manylinux2014_x86_64.whl (322 kB)

322.0/322.0 kB 451.5 MB/s eta 0:00:00

Collecting websockets>=13.0

Downloading websockets-15.0.1-cp311-cp311-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux2014_x86_64.whl (182 kB)

182.3/182.3 kB 262.2 MB/s eta 0:00:00

Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.4.1)

Requirement already satisfied: cffi>=1.12.0 in /usr/local/lib/python3.11/site-packages (from curl_cffi>=0.7->yfinance) (1.15.1)

Collecting certifi>=2024.2.2

Downloading certifi-2025.8.3-py3-none-any.whl (161 kB)

161.2/161.2 kB 235.5 MB/s eta 0:00:00

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2.8.2)

Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2023.3)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.1.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.4)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (1.26.15)

Requirement already satisfied: pycparser in /usr/local/lib/python3.11/site-packages (from cffi>=1.12.0->curl_cffi>=0.7->yfinance) (2.21)

Loading [MathJax]/extensions/Safe.js Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.16.0)

```
Installing collected packages: websockets, protobuf, certifi, curl_cffi, yfinance
  Attempting uninstall: certifi
    Found existing installation: certifi 2022.12.7
    Uninstalling certifi-2022.12.7:
      Successfully uninstalled certifi-2022.12.7
  Attempting uninstall: yfinance
    Found existing installation: yfinance 0.2.54
    Uninstalling yfinance-0.2.54:
      Successfully uninstalled yfinance-0.2.54
Successfully installed certifi-2025.8.3 curl_cffi-0.13.0 protobuf-6.32.0 websockets-
15.0.1 yfinance-0.2.65
```

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.2

[notice] To update, run: `pip install --upgrade pip`

```
In [2]: import datetime
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import yfinance as yfin
import seaborn as sns
```

```
In [3]: # Download stock prices from Yahoo Finance and set the time period for download
start = datetime.date(2018, 1, 2)
end = datetime.date(2023, 12, 31)
stocks = yfin.download(["AAPL", "F", "WMT"], start, end, auto_adjust = False)["Adj
stocks.head()
```

```
[*****100%*****] 3 of 3 completed
```

```
Out[3]:
```

	Ticker	AAPL	F	WMT
	Date			
	2018-01-02	40.380993	8.358681	28.973358
	2018-01-03	40.373947	8.424706	29.226089
	2018-01-04	40.561497	8.569958	29.252539
	2018-01-05	41.023300	8.715214	29.425938
	2018-01-08	40.870937	8.682199	29.860868

```
In [4]: stocks.index = pd.to_datetime(stocks.index).strftime("%Y-%m-%d")
```

```
In [5]: # In order to calculate the returns of the stocks, we need to drop the NA rows.
stocks_returns = stocks[["AAPL", "F", "WMT"]].dropna().pct_change()
stocks_returns = stocks_returns.dropna()
stocks_returns.head()
```

```
Out[5]:
```

	Ticker	AAPL	F	WMT
	Date			
	2018-01-03	-0.000174	0.007899	0.008723
	2018-01-04	0.004645	0.017241	0.000905
	2018-01-05	0.011385	0.016949	0.005928
	2018-01-08	-0.003714	-0.003788	0.014781
	2018-01-09	-0.000115	-0.005323	-0.012007

With the stock returns for Apple, Ford Motor and Walmart, we can calculate the covariance matrix for the 3 stock returns.

```
In [6]: # Calculate covariance matrix for Apple, Ford Motor and Walmart
# Use np.allclose to verify if the matrix is a symmetric matrix
stock_returns_covariance_matrix = stocks_returns.cov()
print("Apple, Ford Motor and Walmart Covariance Matrix:")
print(stock_returns_covariance_matrix)
print("\nIs the covariance matrix symmetric?", np.allclose(stock_returns_covariance
```

Apple, Ford Motor and Walmart Covariance Matrix:

Ticker	AAPL	F	WMT
AAPL	0.000398	0.000194	0.000103
F	0.000194	0.000668	0.000068
WMT	0.000103	0.000068	0.000199

Is the covariance matrix symmetric? True

As we can see from the above covariance matrix for Apple, Ford Motor, and Walmart, the variances of the three stocks are on the diagonal of the matrix. The covariances of any pairs of the stocks are on the off-diagonal of the matrix. This covariance matrix is also a symmetric matrix based on the definition.

1.4. Correlation Matrix According to the previous lesson, correlation is a metric to measure the co-movement of two variables while removing their scales. Without the scale difference, we can compare correlations from different pairs of variables. It is also a symmetric matrix. Let's continue our Apple, Ford Motor, and Walmart stock return example to calculate their correlation matrix.

```
In [7]: # Calculate correlation matrix for Apple, Ford Motor and Walmart
# Verify if a correlation matrix a symmetric matrix
stock_returns_correlation_matrix = stocks_returns.corr()
print("Apple, Ford Motor and Walmart Correlation Matrix:")
print(stock_returns_correlation_matrix)
```

Loading [MathJax]/extensions/Safe.js correlation matrix symmetric?", np.allclose(stock_returns_correlati

Apple, Ford Motor and Walmart Correlation Matrix:

Ticker	AAPL	F	WMT
AAPL	1.000000	0.376461	0.365246
F	0.376461	1.000000	0.185249
WMT	0.365246	0.185249	1.000000

Is the correlation matrix symmetric? True

We can see from the above correlation matrix that it is also a symmetric matrix. The diagonal values are the correlations of 3 stocks to themselves. The off-diagonal values are the correlations of different paired stock returns. Apple's stock return has similar correlation to both Ford Motor and Walmart's stock returns. Ford Motor and Walmart's stock returns have the lowest correlation among other stock return pairs.

1.5. Converting between Covariance and Correlation Matrices From the previous lesson, we learned that covariance and correlation have the following relationship.

$$\text{Corr}(X,Y) = \frac{\text{Cov}(X,Y)}{\sqrt{\text{Var}(X)} * \sqrt{\text{Var}(Y)}}$$

where $\text{Cov}(X,Y)$ is the covariance of X and Y , $\text{Var}(X)$ and $\text{Var}(Y)$ are variances of X and Y .

We can use the above formula to convert between covariance and correlation matrices easily. Let's demonstrate the conversion with the following code.

```
In [8]: # Convert covariance to correlation
def cov_to_corr(cov_matrix):
    d = np.sqrt(np.diag(cov_matrix))
    corr_matrix = cov_matrix / np.outer(d, d)
    return corr_matrix

# Convert correlation to covariance
def corr_to_cov(corr_matrix, std_devs):
    cov_matrix = corr_matrix * np.outer(std_devs, std_devs)
    return cov_matrix
```

```
In [9]: # Use our Apple, Ford Motor and Walmart stock returns example to demonstrate the co
# Calculate standard deviations of stock returns
stocks_returns_std = stocks_returns.std()

#convert covariance to correlation example
cov_to_corr(stock_returns_correlation_matrix)
```

```
Out[9]:
```

Ticker	AAPL	F	WMT
AAPL	1.000000	0.376461	0.365246
F	0.376461	1.000000	0.185249
WMT	0.365246	0.185249	1.000000

```
In [10]: #convert correlation to covariance example
corr_to_cov(stock_returns_correlation_matrix, stocks_returns_std)
```

```
Out[10]:
```

Ticker	AAPL	F	WMT
AAPL	0.000398	0.000194	0.000103
F	0.000194	0.000668	0.000068
WMT	0.000103	0.000068	0.000199

2. Triangular Matrix

Another commonly used matrix is the triangular matrix. A triangular matrix is a square matrix where all the entries on one side of the main diagonal are zero. There are two types of triangular matrices: lower triangular matrix and upper triangular matrix.

2.1. Lower Triangular Matrix A **lower triangular matrix** is a square matrix whose entries above the main diagonal are zero. The main diagonal and entries below it can be any value. Below is a demonstration of a lower triangular matrix.

$$B = \begin{bmatrix} m & 0 & 0 \\ a & n & 0 \\ b & c & p \end{bmatrix}$$

2.2. Upper Triangular Matrix

An **upper triangular matrix** is a square matrix whose entries below the main diagonal are zero. The main diagonal and entries above it can be any value. Below is a demonstration of an upper triangular matrix.

$$B = \begin{bmatrix} m & a & b \\ 0 & n & c \\ 0 & 0 & p \end{bmatrix}$$

2.3. Properties of a Triangular Matrix Here are some key properties of a triangular matrix.

1. The determinant is the product of the diagonal elements.
2. The inverse of a triangular matrix (if it exists) is also triangular.
3. The product of two upper (or lower) triangular matrices is upper (or lower) triangular.

The above properties are all very easy to prove using matrix operations, so we will leave these proofs to the readers.

3. Symmetric Positive Definite (PD) Matrices and Symmetric Positive Semi-Definite (SPD) Matrices

In this section, we are going to introduce two important types of matrices: symmetric positive definite (PD) matrices and symmetric positive semi-definite matrices (SPD).

3.1. Symmetric Positive Definite (PD) Matrices A symmetric matrix A is positive definite if $x^T A x > 0$ for all non-zero vectors x .

Here are some key properties of a symmetric positive definite matrix:

1. All eigenvalues of the matrix are positive
2. The determinant of the matrix is positive
3. The matrix has full rank
4. The matrix is invertible
5. The summation and multiplication of two symmetric positive definite matrices is also a symmetric positive definite matrix

These properties of symmetric positive definite matrices—full rank, positive determinant, and positive eigenvalues—are intimately connected. These characteristics make symmetric positive definite matrices particularly useful in various mathematical and applied contexts, providing guarantees of non-singularity, stability, and convergence in many algorithms and methods.

Usually, a quick way to identify if a symmetric matrix is positive definite is to calculate its determinant and check if the determinant is positive. For example, let's take a look at the following symmetric matrix.

$$\begin{bmatrix} 3 & 4 \\ 4 & 5 \end{bmatrix}$$

This is a symmetric matrix. However, the determinant is $(3 \times 5) - (4 \times 4) = -1$, this matrix is not positive definite.

3.2. Symmetric Positive Semidefinite (SPD) Matrices A symmetric matrix A is positive semidefinite if $x^T A x \geq 0$ for all vectors x .

Here are the key properties of a symmetric positive semidefinite matrix:

1. All eigenvalues of the matrix are non-negative
2. The determinant of the matrix is non-negative
3. The matrix may not have full rank
4. The matrix is not always invertible

We usually also use the determinant of a symmetric matrix to quickly check if it is also a positive semidefinite. For the following matrix example,

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

we can check that the determinant is 0. Hence, this matrix is a positive semidefinite matrix.

Going back to covariance matrices and correlation matrices, they are both symmetric positive semidefinite but not necessarily symmetric positive definite. The following Python code can be used to check if a matrix is a symmetric positive definite or a symmetric positive semidefinite.

```
In [11]: def is_positive_definite(matrix):
          return np.all(np.linalg.eigvals(matrix) > 0)

A = stock_returns_correlation_matrix

print("Is A positive definite?", is_positive_definite(A))
print("Eigenvalues:", np.linalg.eigvals(A))
print("Determinant:", np.linalg.det(A))
print("Rank:", np.linalg.matrix_rank(A))
```

```
Is A positive definite? True
Eigenvalues: [1.62523875 0.55989698 0.81486427]
Determinant: 0.7414990015150257
Rank: 3
```

```
In [12]: def is_positive_semidefinite(matrix):
          return np.all(np.linalg.eigvals(matrix) >= 0)

A = stock_returns_correlation_matrix

print("Is A positive semidefinite?", is_positive_semidefinite(A))
print("Eigenvalues:", np.linalg.eigvals(A))
print("Determinant:", np.linalg.det(A))
print("Rank:", np.linalg.matrix_rank(A))
```

```
Is A positive semidefinite? True
Eigenvalues: [1.62523875 0.55989698 0.81486427]
Determinant: 0.7414990015150257
Rank: 3
```

4. Cholesky Factorization

4.1. Definition of Cholesky Factorization

Cholesky factorization is a method to decompose a symmetric matrix. Cholesky

states that a symmetric positive definite matrix S can be decomposed as

follows:

$$S = LL^T$$

where L is a lower triangular matrix or a Cholesky factor.

Let's look at a numeric example. Suppose we have the following 3×3 symmetric positive definite matrix.

$$S = \begin{bmatrix} s_{11} & s_{21} & s_{31} \\ s_{21} & s_{22} & s_{32} \\ s_{31} & s_{32} & s_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} = LL^T$$

$$\Downarrow$$

$$LL^T = \begin{bmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{31}l_{11} & l_{31}l_{21} + l_{32}l_{22} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{bmatrix}$$

From the above matrices, we can get the following formulae for the lower triangular matrix.

$$\begin{aligned} l_{11} &= \sqrt{s_{11}} \\ l_{21} &= \frac{s_{21}}{l_{11}} \\ l_{31} &= \frac{s_{31}}{l_{11}} \\ l_{22} &= \sqrt{s_{22} - l_{21}^2} \\ l_{32} &= \frac{s_{32} - l_{21}l_{31}}{l_{22}} \\ l_{33} &= \sqrt{s_{33} - l_{31}^2 - l_{32}^2} \end{aligned}$$

To get the above solution, you must start to solve the first row of L . Then, you move on to the second row and the next row until you solve all the elements in L .

Let's look at a numeric example. Suppose we have the following symmetric positive definite matrix (please verify), $S = \begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix}$

Let's find the Cholesky factor using the above equations we derived.

$$\begin{aligned} l_{11} &= \sqrt{25} = 5 \\ l_{21} &= \frac{15}{5} = 3 \\ l_{31} &= \frac{-5}{5} = -1 \\ l_{22} &= \sqrt{18 - 9} = 3 \\ l_{32} &= \frac{0 - (3)(-1)}{3} = 1 \\ l_{33} &= \sqrt{11 - (-1)^2 - (1)^2} = 3 \end{aligned}$$

$$L = \begin{bmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 3 \end{bmatrix}$$

Python Numpy has a function to calculate the Cholesky factor for a matrix. We can use it to verify our manual calculation.

```
In [13]: S = np.array([[25,15,-5],
                        [15,18,0],
                        [-5,0,11]])

L_numpy = np.linalg.cholesky(S)
print("\nNumPy Cholesky factor:")
print(L_numpy)
```

NumPy Cholesky factor:

```
[[ 5.  0.  0.]
 [ 3.  3.  0.]
 [-1.  1.  3.]]
```

Cholesky factorization algorithm greatly improves the computer efficiency to solve for linear equations. It can also be used to compute the inverse of a matrix more efficiently than some other methods. Cholesky factorization is also numerically stable, making it useful for computations involving ill-conditioned matrices.

4.2. Using Cholesky Factorization for Monte Carlo Simulation

Cholesky factorization is useful in Monte Carlo simulations or other optimization tasks. In this section, we will use Cholesky factorization in a Monte Carlo simulation to generate correlated random variables. Let's use the Apple and Ford Motor stock return correlation matrix as an example. The process involves the following steps:

1. Compute the Cholesky factorization of the stock return correlation matrix to get the factor L.
2. Generate independent standard normal random variable Z.
3. Compute $X = LZ$ to get correlated random variables.

First, let's calculate the correlation of Apple and Ford Motor stock returns.

```
In [14]: # Create a new dataframe with just Apple and Ford Motor stock returns from the data
two_stock_returns = stocks_returns[["AAPL", "F"]]
original_correlation = two_stock_returns.corr()
original_correlation
```

```
Out[14]: Ticker    AAPL    F
         Ticker
         AAPL  1.000000  0.376461
         F    0.376461  1.000000
```

Now, let's implement the simulation using Cholesky factorization as described above.

```
In [15]: # Create a function to generate random samples from correlated variables
def generate_correlated_samples(n_samples, correlation_matrix):
    # Compute Cholesky factorization
    L = np.linalg.cholesky(correlation_matrix)

    # Generate independent standard normal samples
    Z = np.random.standard_normal((correlation_matrix.shape[0], n_samples))

    # Generate correlated samples
    X = L @ Z

    return X

# Generate samples
n_samples = 10000
X = generate_correlated_samples(n_samples, original_correlation)
X = pd.DataFrame(generate_correlated_samples(n_samples, original_correlation).T, columns=['AAPL', 'F'])
# Compute sample correlation
sample_correlation = X.corr()
```

Our next step is to visualize the result from original data and sampled data using a Monte Carlo simulation.

```
In [16]: # Visualize results
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

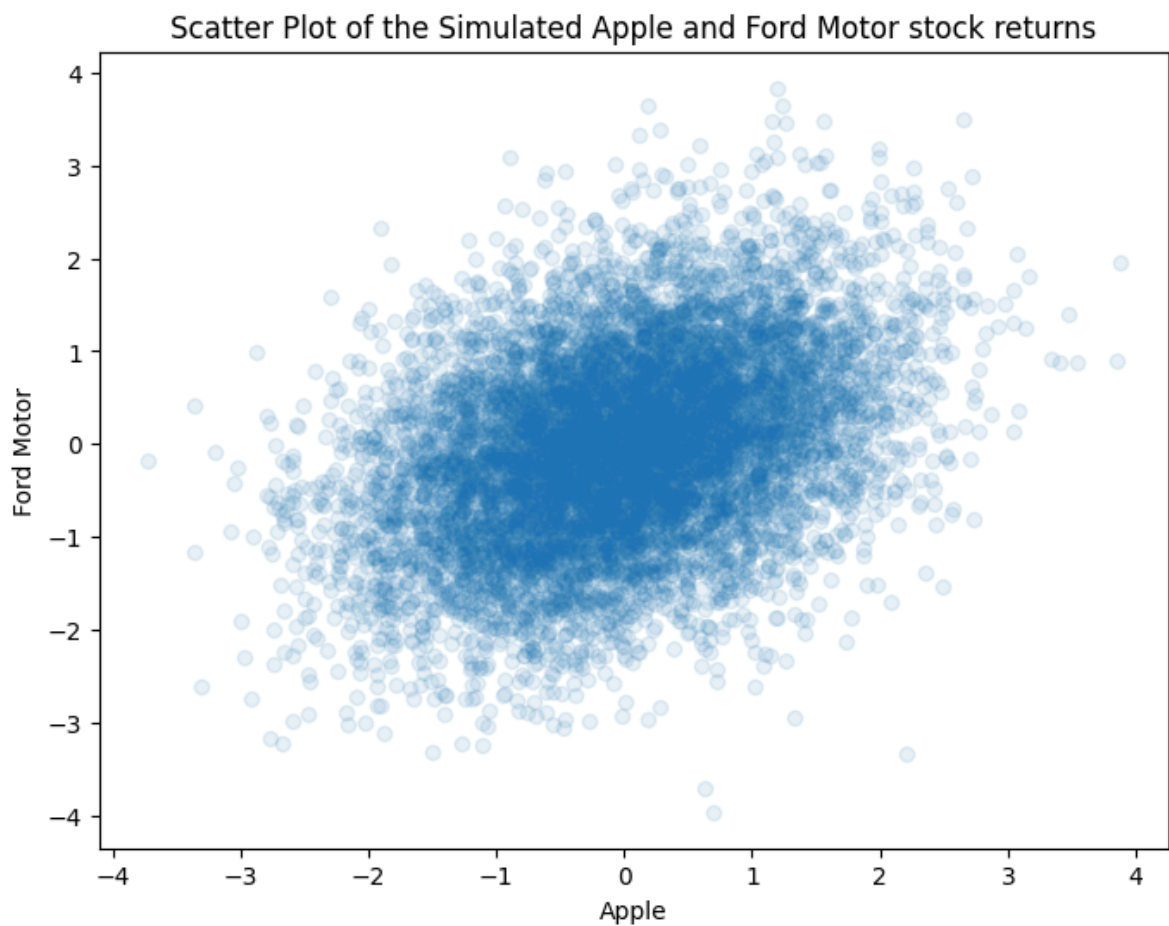
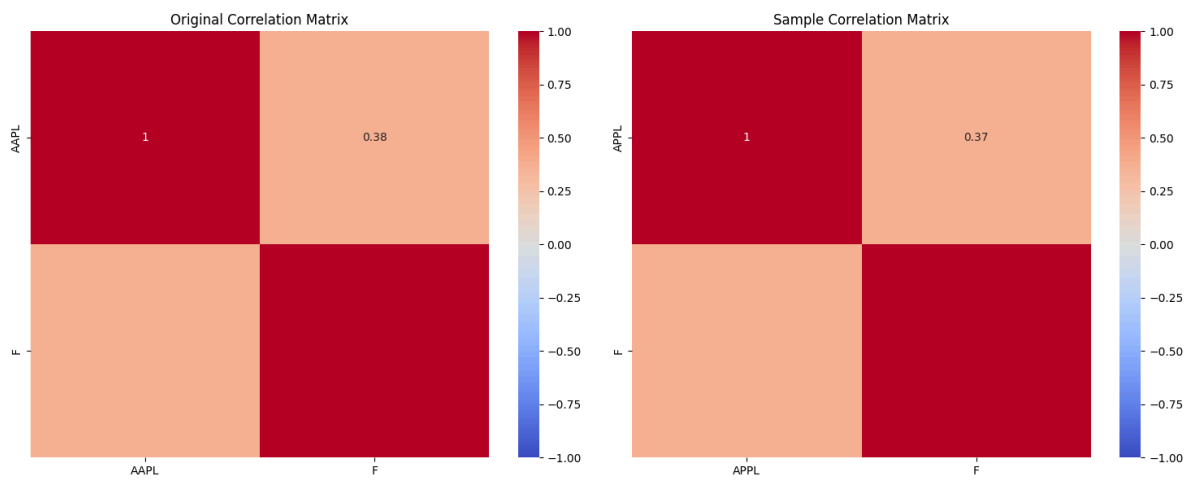
sns.heatmap(original_correlation, annot=True, cmap='coolwarm', vmin=-1, vmax=1, ax=ax1)
ax1.set_title('Original Correlation Matrix')
ax1.set_xlabel='', ylabel='')

sns.heatmap(sample_correlation, annot=True, cmap='coolwarm', vmin=-1, vmax=1, ax=ax2)
ax2.set_title('Sample Correlation Matrix')
ax2.set_xlabel='', ylabel='')

plt.tight_layout()
plt.show()

# Scatter plot of the simulated Apple and Ford Motor stock returns
plt.figure(figsize=(8, 6))
plt.scatter(X["AAPL"], X["F"], alpha=0.1)
plt.xlabel('Apple')
plt.ylabel('Ford Motor')
plt.title('Scatter Plot of the Simulated Apple and Ford Motor stock returns')
```

```
plt.ylabel('Ford Motor')
plt.show()
```



From the above original correlation matrix and sample correlation matrix, we can see that the correlation values of the original data and the simulated data are very close.

5. Conclusion In this lesson, we introduced a few special matrices and some linear transformations related to these matrices. We started with the definition and properties of a

symmetric matrix and then its application, especially its diagonalized form. Then, we described what upper and lower triangular matrices are. We moved to symmetric positive definite matrices followed by symmetric positive semidefinite matrices. We finished the lesson by introducing Cholesky factorization. We provided the definition and went through one numeric example. We ended the section with a Cholesky factorization application with a Monte Carlo simulation. All these special matrices will play important roles in the financial analysis tools we are going to learn in future lessons.

References

- Axler, Sheldon. *Linear Algebra Done Right*. 4th edition, Springer, 2024.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Grammatikos, Theoharry, and Nikolaos I. Papanikolaou. "Applying Benford's Law to Detect Accounting Data Manipulation in the Banking Industry." *Journal of Financial Services Research*, vol. 59, 2021, pp. 115-142, <https://link.springer.com/article/10.1007/s10693-020-00334-9>.

- **Required pages:** Skim tables, charts, and graphs
- **Estimated reading time:** 20 minutes

1. Chang, Kuo-Chung. "The Effect of Data Theft on a Firm's Short-Term and Long-Term Market Value." *Mathematics*, vol. 8, no. 5, 2020, <https://www.mdpi.com/2227-7390/8/5/808/htm>.

- **Required pages:** pp. 6–7
- **Estimated reading time:** 10 minutes

1. Liu, Grace. "Data Quality Problems Troubling Business and Financial Researchers: A Literature Review and Synthetic Analysis." *Journal of Business & Finance Librarianship*, 2020, https://digitalcommons.wcupa.edu/lib_facpub/13/.

- **Required pages:** pp. 29–36
- **Estimated reading time:** 25 minutes

ETHICAL PRACTICES WITH DATA

Reading Time	30 minutes
Prior Knowledge	FD Module 1, Types of Data, attributes of data, best practices with data
Keywords	flash crash, fiduciary, smoothing data

Not all data is created in an ethical, straightforward, complete, and forthright manner. Just as we need to apply critical thinking to assess the quality, accuracy, and completeness of data, we should also use good ethical practices when we provide data to others or use it in our own models. This lesson will consider a dozen unethical practices associated with data.

Ethics are an important part of finance. Firms like to include ethics in training their employees. Indeed, one of the largest accounting firms in the world administered an ethics exam to its employees. It turns out that the employees cheated on their ethics exam! The firm itself failed to disclose this information to the regulatory board (GaNun).

This lesson deals with ethical issues as they apply to the handling of data. Each subsection describes one of the unethical ways people handle data inappropriately: investing, exaggerating, distorting, inappropriately collecting, hiding, misconstruing, etc. Each includes at least one real-world example of behavior undertaken by an individual, company, or even a market.

1. Intentionally Creating Misleading Data

One day, an artist named Simon Weckert collected 99 smart phones (Barret). He put them into a wagon that he then proceeded to walk along the streets of Munich, ironically the street near Google headquarters in Munich—he was hacking the Google Maps app! About an hour into his walk, Google Maps showed long, snarling traffic—despite there not being any cars in the area. The artist claims he was pointing out the simplicity behind these systems. There was really no harm done in this clever prank. This makes us question our assumptions when we receive data, like traffic conditions.

If this exercise were done in financial markets, it would be an entirely different story. It is unethical to intentionally create misleading data. Just as we may automatically follow the conditions on a traffic app, a RoboAdvisor may automatically follow the conditions in the marketplace (D'Acunto and Rossi). They may react without questioning data because they focus on speed and efficiency rather than authenticating the results.

One egregious way to intentionally mislead a market is to spoof it. One such person who did that in 2010 was Navinder Singh Sarao. He was held responsible for initiating a market downturn known as the Flash Crash of 2010. Sarao pled guilty to spoofing the market and paid with a prison sentence and fines.

What is spoofing? Spoofing is an extreme type of bluff. Spoofing occurs when a trader places a large order that he has no intention of getting filled. Instead, the intention is to manipulate the market in one's favor. For example, a spoofer placing a large buy order would place a huge bid order well below the current market bid price. When market makers and other traders, including algorithmic traders, sense a motivated buyer, they pull their ask prices and raise them. These price increases trigger other traders to do the same. Yet, this was the spoofer's motive all along—a manipulation of price in the desired direction.

In Mr. Sarao's case, he spoofed the market by placing a large sell order (above the ask price) for the E-mini S&P futures contract. His order caused the algorithms to lower their prices, which in turn caused other traders and algorithms to lower their prices, and so on. Prices plummeted. The actions and reactions caused the Flash Crash, an intraday market crash that wiped out more than \$1 trillion in market value. Other markets did not exhibit the drop, and so the market learned that this was a one-off accident. Within the hour, most of the market corrected.

It is unethical to crate misleading data. Markets trade in real time and can be misled by spoofers. Fortunately, regulation and greater controls exist at many exchanges today that minimize the reoccurrence of a trader spoofing the market and causing another flash crash. It is still a criminal offense to spoof the market.

2. Propagating Fake Reviews

Next time you're in London, you may want to dine at what was once the top-rated restaurant on tripadvisor.com: The Shed (Butler). With pictures of dishes, an outside garden, and dozens of reviews, The Shed seemed to be the trendy place to eat. The only problem is that the restaurant didn't exist. The initial reviews were all fake. Ironically, The Shed was the go-to place. Phone calls into the restaurant only made it to the answering machine. When people realized how trendy the restaurant was, they wrote reviews even though they never even went there. Part of the momentum of going viral was hiring celebrities that posed with dishes (that weren't even from the restaurant) to lend an air of credibility and panache.

Reading reviews is not the same as performing due diligence. As more details of the FTX scandal become public, some celebrities are coming forward to admit that they were investors. Many people are familiar with the movie star Kevin Bacon. He was an investor in FTX and claims he lost millions of dollars. Even celebrities can become victims of the halo effect or appeal to authority. Marketers know the power of celebrity endorsements, to the point where many athletes and musicians earn millions more from product endorsements than they earn in wages.

Do you wonder how many restaurant reviews are fake? There are parties with vested interests who are tempted to post glowing review after glowing review. A quick click into that reviewer shows they reviewed nothing else. The same can be said for forums that review stocks. Some people join these popular forums to tout stocks (or to bash them if they have short positions). Good ethics avoids misrepresentation.

3. Failing to Disclose Fees or Conflicts of Interest

According to a data survey by the data research company Morning Consult, only 36% of adults trusted investment and wealth managers. The rest either did not trust them or had no opinion about them. Part of the problem is that financial advisors fail to disclose fees or reveal conflicts of interest. For example, an advisor may recommend a mutual fund A over B for a client. Although B may be more suitable for the investment profile and risk tolerance, the advisor is paid a much better commission for selling mutual fund A. Failing to disclose fees or reveal conflicts of interest violates a fiduciary trust.

In the investment world, these are criminal activities. For example, two robo-advisors, one which had more than \$11 billion in client money, made false statements about a strategy involving tax-loss harvesting (U.S. Securities and Exchange Commission, "SEC Charges Two Robo-Advisers"). These false disclosures misled investors into thinking the funds had superior performance (higher returns, less risk, smaller fees) and naturally makes them appear more attractive than other funds. Other times, managers fail to disclose fees (U.S. Securities and Exchange Commission, "SEC Charges Investment Adviser").

Good ethics show all data associated with investment: historical results, fees, and risks. Regulation helps. Some industries protect the consumer/investor better than others.

4. Using Simulated Data Instead of Real Data without Explicit Disclosure

Another way to lie with data is to use simulated data while trying to pass it off as real data. For example, a hedge fund raised millions of dollars by falsely claiming that it had a lengthy track record based on actual trades using real money (U.S. Securities and Exchange Commission, "SEC Charges Father-and-Son Hedge Fund Managers"). In reality, the results were from back-tested hypothetical simulations. Simulations have details that can skew a study one way or another. For example, a simulation of a back-test may ignore a bid-ask spread and market impact. Misrepresenting the type of data is illegal, and the managers agreed to pay millions to settle the fraud case. Using simulated returns in and of itself is not illegal or unethical. When using simulated data, you should clearly indicate that the results are not real, but merely a product of back-tests or scenarios. Even when a fund has a track record going back years, a disclaimer is added that past performance is no guarantee of future results. Misleading is lying.

5. Adding Good Data to Bad Data

Student cheating is well known, but there is also something called teacher cheating. This story was described in Levitt and Dubner's book *Freakanomics: A Rogue Economist Explores the Hidden Side of Everything*. To make their own teaching appear more effective, some teachers in Chicago cheated by adding points to students' tests. When students submitted their test papers, the teachers would fill in the last few questions correctly, as most students didn't attempt to answer the hard questions due to lack of time or difficulty. So these teachers could take a student's exam paper and spend a few seconds improving the score. The teachers did not spend any time erasing answers, but did spend time on filling in a memorized pattern of multiple-choice responses for the last few questions, which most students never attempted. Overall, the teacher cheating seemed to work: the scores of students improved! But there were some inconsistent statistics about performance that raised red flags. Many students incorrectly answered easy and medium questions yet got a sequence of difficult questions correct. Moreover, nearly all the students in a classroom got the same exact difficult questions correct. The coincidences were astounding. Further investigation showed there was an incentive for teachers: improved test scores meant more money for teachers.

In banking, the stakes are higher, and the methods are much more sophisticated. Benford's Law is used to detect accounting data manipulation in the banking industry. Benford's law predicts the relative frequencies of digits. Please read the paper by Grammatikos and Papanikolaou to learn how Benford's Law was applied to detect accounting data manipulation in the banking industry. One might think a small percentage of good data would go unnoticed. Good ethical practice is to not interfere with data quality.

6. Hiding Data

Instead of adding good data to bad data, why not hide some data altogether? That's what was done in the case of some public debt. Horn describes the systematic underreporting of public debt. The study includes data from 50 years. In strong economies, hiding debt is easy. However, in bad economic times, hidden debts are likely to be exposed. Hiding debt can make politicians, comptrollers, or budgets look better than they are, and politicians may make promises that can help get them re-elected but won't make good on them when the true revenues and expenses are tallied.

Good intentions may cause people to hide data. If you were removing outliers from a distribution, that may seem to be reducing noise, improving the overall results. Yet, what one person considers to be an outlier, the other may consider to be an important extreme case. Likewise, on a "thin trading day" (a day with very little liquidity), one may consider the bid-ask spread to be non-representative of overall liquidity. However, these illiquid periods need to be included in models that measure liquidity and overall risk.

Other times, data may be biased and skew results. Suppose a restaurant provides a feedback form only to repeat customers. Presumably, a repeat customer enjoyed their experience and revisits the establishment. However, what about a one-time customer who did not have a good experience? They are denied the chance to receive a feedback form in the first place. Of course, many online websites (such as Yelp) offer the opportunity for dissatisfied customers to express their views.

7. Smoothing Data

Smoothing sounds like a good thing. In Econometrics, you will smooth time series to filter out noise. However, smoothing profit-and-loss (P and L) makes data appear less risky than it really is. Smoothing under-reports volatility. Underreporting volatility is a common phenomenon at alternative funds or funds that trade in securities that don't have clear, liquid, mark-to-market pricing (Le Marois). Granted, if a security hasn't traded in months or is difficult to value because of a lack of fungibility, it is hard to provide accurate marks. If volatility is understated, then Sharpe ratios are overstated. For example, suppose a strategy earns the fund manager the following daily returns: +5, -1, +1, -3. The manager could turn the +5 into a +1, holding back a reserve of 4. That reserve could flatten the -3 entirely and mitigate the -2 to a -1, so instead of having swings of up 5 and down 3, the strategy reports +1, 0, +1, 0. This looks very good: small wins and no losses. Taken to scale, holding back gains for offsetting losses will lower the volatility of a strategy. Good ethics avoids smoothing.

8. Extrapolating Data

Extrapolating data is a problem. Some models have linear regions and then quickly change to non-linear. Think market impact. Previously in this course, we saw the problem associated with the Space Shuttle Challenger disaster. Sadly, the analysis shows that the 20-degree decrease in temperature (which reflects the temperature conditions on launch day) increases the odds of failing by 15-fold. Good ethical practice is to ensure predicted results are not outside the region of testing. See the article ByUiStats here: <https://byuistats.github.io/Statistics-Notebook/Analyses/Logistic%20Regression/Examples/challengerLogisticReg.html>

The danger of extrapolating is assuming that the trend will continue outside the region of predictors where it was tested. If a system is linear, then extrapolation would not create problems. The challenge of non-linearity makes it dangerous to assume trends continue outside the test region.

9. Stealing Data

Companies have data stolen. Some of this data may contain important customer information, which is used to hack into their accounts, draining them of funds or setting up new identities in their names. In other cases, it is copyrighted, patented, or trademarked. In previous lessons, we saw employees who took data (and code) from companies. Read Chang et al. to see the effect of data theft on a firm's market value.

10. Data Quality Problems

You've heard the acronym GIGO: Garbage In, Garbage Out. Yet, how do we know we have garbage in the first place? Although not a straitjacket ethical issue, some data is of poor quality. If you knew data had quality problems but still modeled it extensively and reported on the results, you propagate the quality issue. A model's rigor and formality does not improve the quality of data that is questionable from the start (see Liu for more info).

One way data quality issues are avoided is by requiring audited financial statements. The scrutiny of a professional accounting firm provides reliability and trust in the reported results. Consider the case of the world's largest rental car company, Hertz. Several years ago, the SEC charged Hertz with inaccurate data reporting (U.S. Securities and Exchange Commission, "SEC Charges Hertz"). Hertz itself had to pay a fine. But what about the institutional investors who made investment decisions about Hertz based on this information? Clearly, the news about the misrepresentation was not known at the time. Those with a fiduciary responsibility should take great care to assess the quality of data they have before they build models and make decisions.

11. Collecting Data without Permission

Smartphone apps like data. Many of them collect our data, even without our permission. There is now a field focused on data privacy laws that protect consumers, app users, etc. from companies collecting and reselling their data (Klosowski). The problem is that this data can be used in ways the user never intended.

Good ethics gets explicit permission to store user data. This disclosure is usually part of a license agreement but can get easily get lost in the fine print.

12. Disclosing Data without Permission

Disclosing data without permission is even worse. This means your usage data is sent to companies and third parties without your permission or knowledge. Moreover, what if there's a data breach? There have been major companies whose data banks have been hacked, making people's sensitive information available to criminals.

As you can see, data has its own set of ethics. With more types of data available (think of all the alternative data used in finance), the increased number of accounts people have (with banks, brokerages, exchanges, advisors, apps, etc.), there is more data than ever.

13. Conclusion

Just as we ended the FM course discussing ethics, we end the FD course discussing ethics around data. We repeat that reputations are slow to build on the positive side but quick to tumble on the negative side. Decades of diligent, ethical practice can be outweighed by a single act of unethical behavior. Unfortunately, the internet has no ability to forget. Thanks to Instagram pictures, Facebook posts, blogs, YouTube videos gone viral, and news articles that never disappear, stories of unethical behavior have no expiration date.

The stories we have heard—insider trading, fabricating emails, spoofing markets—have turned relatively obscure people into internet memes. Google searches reveal thousands of pages on these people who are known to the world for their infamous act. The opportunity cost is quite high. Many people will avoid working with or hiring these people because of a known track record of dishonesty and deceit. Practice good ethics—in data and in action.

References

Barret, Brian. "An Artist Used 99 Phones to Fake a Google Maps Traffic Jam." *Wired*, 3 Feb. 2020, <https://www.wired.com/story/99-phones-fake-google-maps-traffic-jam/>.

Butler, Oobah. "I Made My Shed the Top Rated Restaurant On TripAdvisor." *Vice*, 6 December 2017, <https://www.vice.com/en/article/434gqw/i-made-my-shed-the-top-rated-restaurant-on-tripadvisor>.

Chang, Kuo-Chung, et al. "The Effect of Data Theft on a Firm's Short-Term and Long-Term Market Value." *Mathematics*, vol. 8, no. 5, 2020, <https://www.mdpi.com/2227-7390/8/5/808/htm>.

D'Acunto, Francesco, and Alberto Rossi. "Robo-advice: An Effective Tool to Reduce Inequalities?" *Brookings*, 5 Oct. 2022, <https://www.brookings.edu/research/robo-advice-an-effective-tool-to-reduce-inequalities/>.

GaNun, Jacqueline. "Accounting Giant Ernst Young Admits its Employees Cheated on Ethics Exams." *NPR*, 28 June 2022, <https://www.npr.org/2022/06/28/1108044858/accounting-giant-ernst-young-admits-its-employees-cheated-on-ethics-exams>.

Grammatikos, Theoharry, and Nikolaos I. Papanikolaou. "Applying Benford's Law to Detect Accounting Data Manipulation in the Banking Industry." *Journal of Financial Services Research*, vol. 59, 2021, pp. 115–142, <https://link.springer.com/article/10.1007/s10693-020-00334-9>.

Horn, Sebastian, et al. "Systematic Underreporting of Public Debt Statistics: 50 Years of Evidence and Recent Progress." *World Bank Blogs*, 3 March 2022, <https://blogs.worldbank.org/developmenttalk/systematic-underreporting-public-debt-statistics-50-years-evidence-and-recent>.

Klosowski, Thorin. "The State of Consumer Data Privacy Laws in the US (And Why It Matters)." *The New York Times*, 6 Sept. 2021, <https://www.nytimes.com/wirecutter/blog/state-of-privacy-laws-in-us/>.

Le Marois, Oliver, and Raphael Douady. "Return Smoothing Practices." *The Hedge Fund Journal*, no. 30, 2007, <https://thehedgefundjournal.com/return-smoothing-practices/>.

Levitt, Steven D., and Stephen J. Dubner. *Freakanomics: A Rogue Economist Explores the Hidden Side of Everything*. William Morrow, 2005.

Liu, Grace. "Data Quality Problems Troubling Business and Financial Researchers: A Literature Review and Synthetic Analysis." *Journal of Business & Finance Librarianship*, November 2020, https://digitalcommons.wcupa.edu/lib_facpub/13/.

U.S. Securities and Exchange Commission. "SEC Charges Investment Adviser with Failing to Disclose Conflicts Arising from Mutual Fund and Account Recommendations." March 2022, <https://www.sec.gov/litigation/litreleases/2022/lr25340.htm>.

U.S. Securities and Exchange Commission. "SEC Charges Father-and-Son Hedge Fund Managers Who Agree to Pay \$4.8 Million to Settle Fraud Case." 2012, <https://www.sec.gov/news/press-release/2012-2012-71.htm>.

U.S. Securities and Exchange Commission. "SEC Charges Hertz with Inaccurate Financial Reporting and Other Failures. 1 Feb. 2019, <https://www.sec.gov/enforce/33-10601-s>.

U.S. Securities and Exchange Commission. "SEC Charges Two Robo-Advisers With False Disclosures." 2018, <https://www.sec.gov/news/press-release/2018-300>.